Convolutional Neural Networks

Vladislav Goncharenko

girafe ai

MIPT, fall 2025

Revise

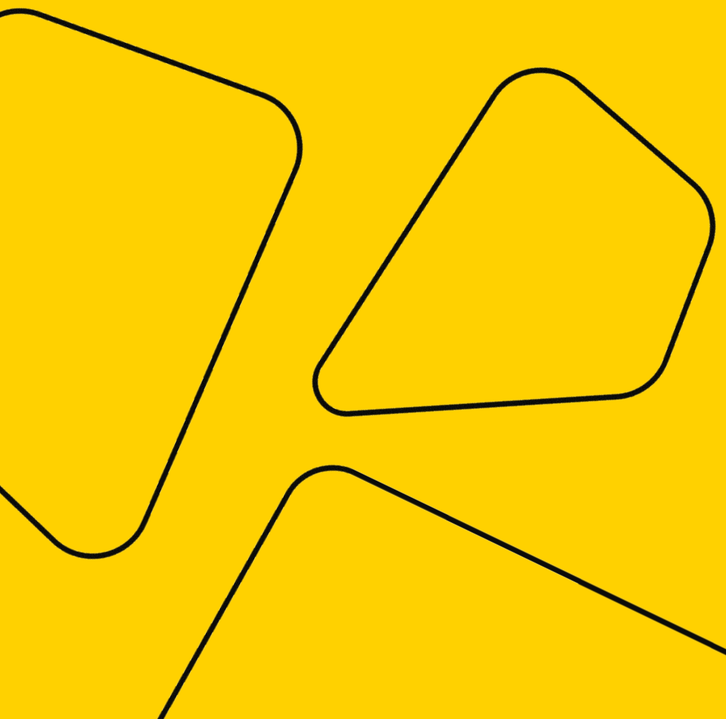
Lecture 10:
Recurrent Neural Networks



1. Vanilla RNN
2. Gradient related problems
    a. Exploding
    b. Vanishing
3. LSTM
    a. GRU
4. Multilayer RNNs
    a. bidirectional

Outline

Lecture 11:
Convolutional Neural Networks



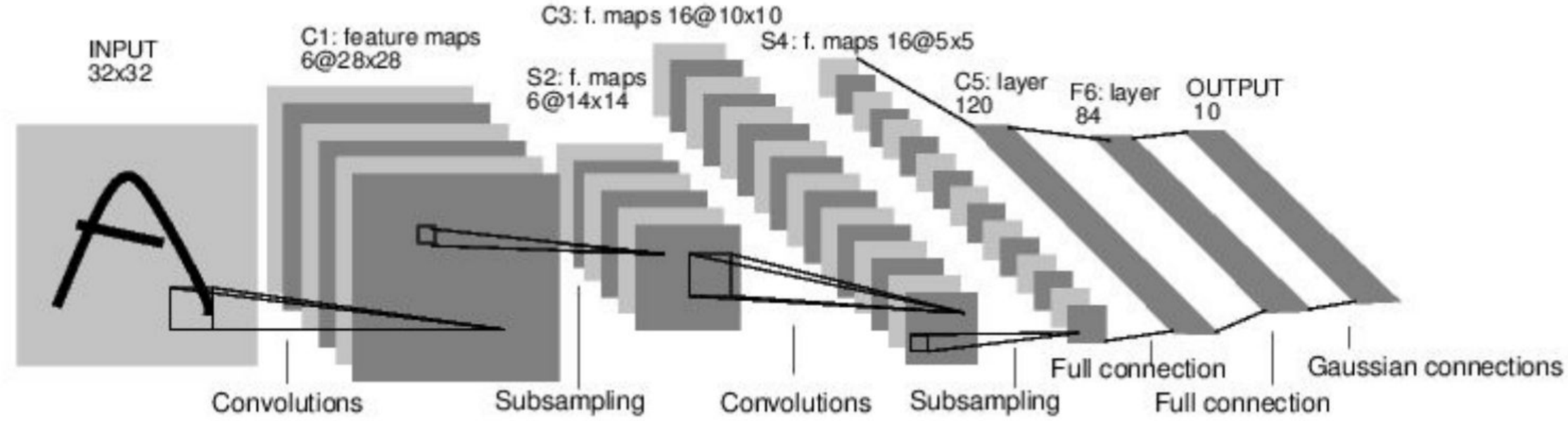
1. Convolutional layer
    a. hyperparameters
    b. receptive field
    c. 1x1 conv
    d. memory consumption
2. CNN filter analysis
3. Pooling layers
    a. Adaptive poolings
4. Classification architectures overview
    a. LeNet
    b. AlexNet
    c. VGG
    d. Inception
    e. ResNet
    f. SENet
    g. EfficientNet

Convolution operation

girafe ai

01

Convolutions in Neural Networks



LeNet-5, LeCun 1998. Article pdf. Wikipedia page.

Convolutional networks were the first hyped in 2012.
They started Deep Learning era which continues up to today.

Image convolution

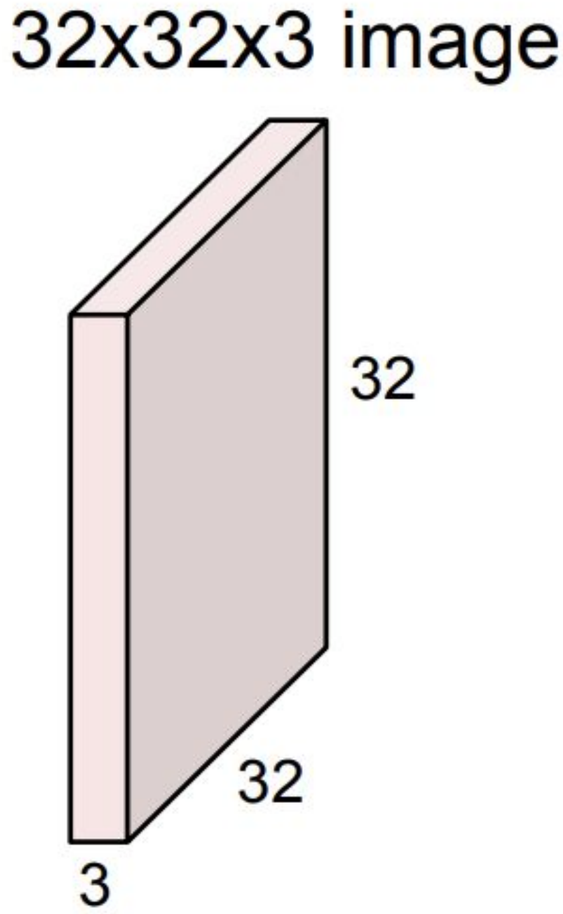
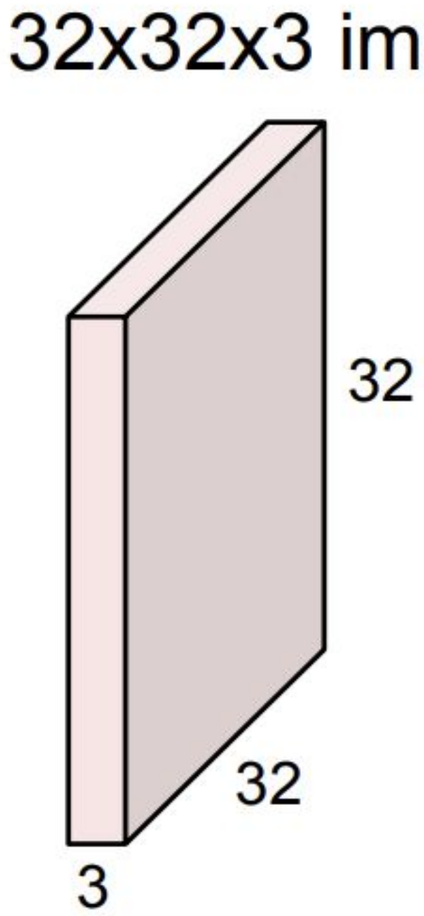
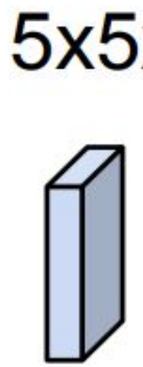


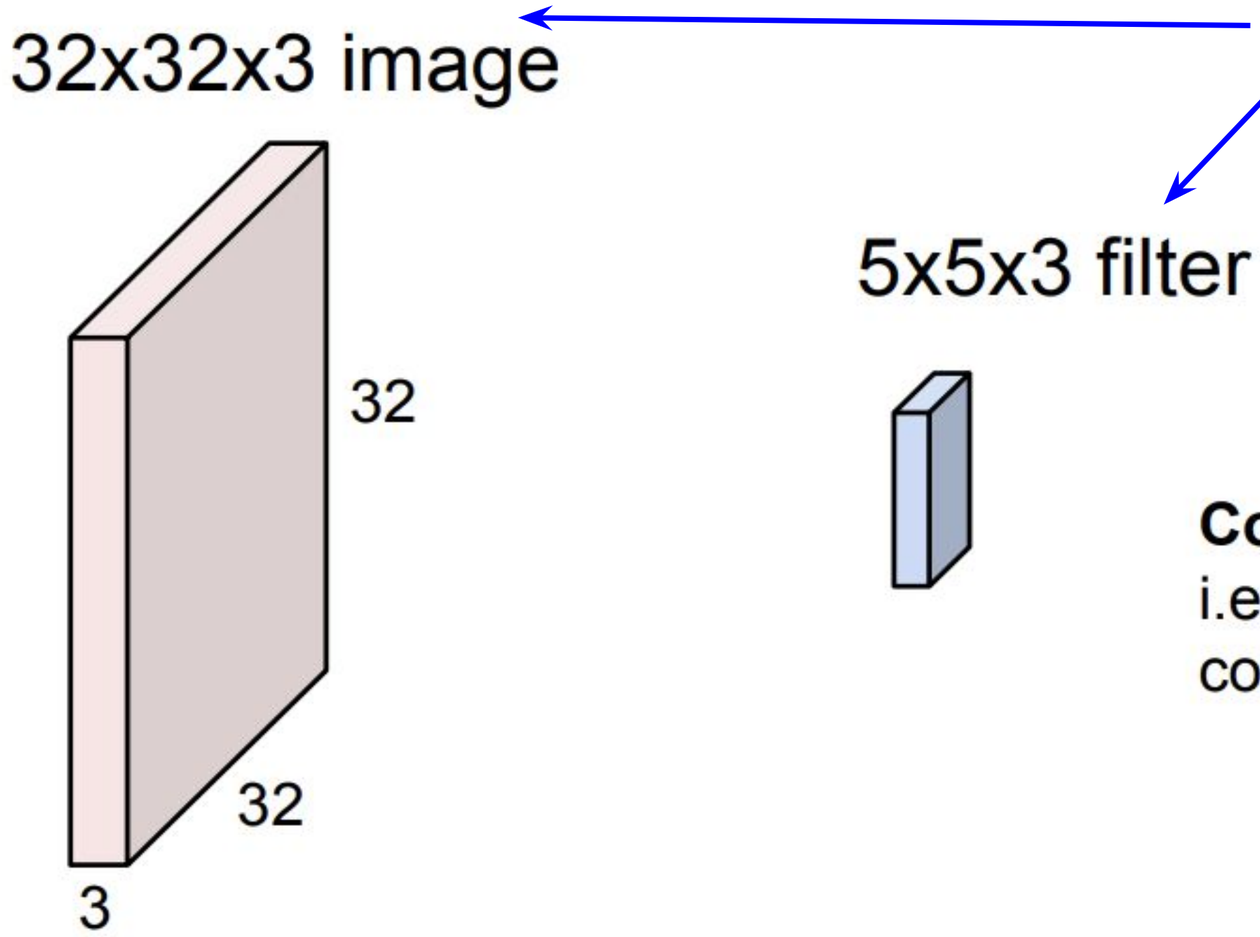
Image convolution





Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

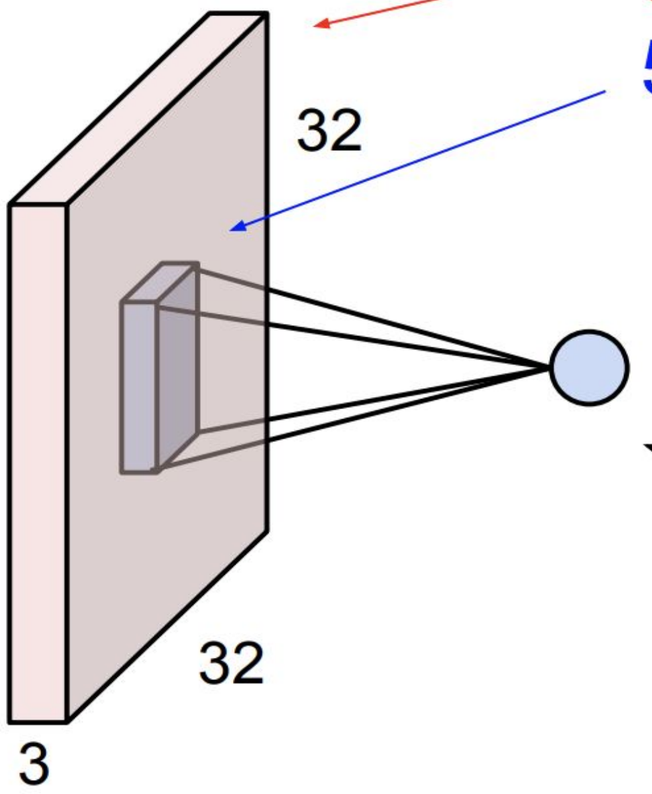
Image convolution



Filters extend the depth of the original image

Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

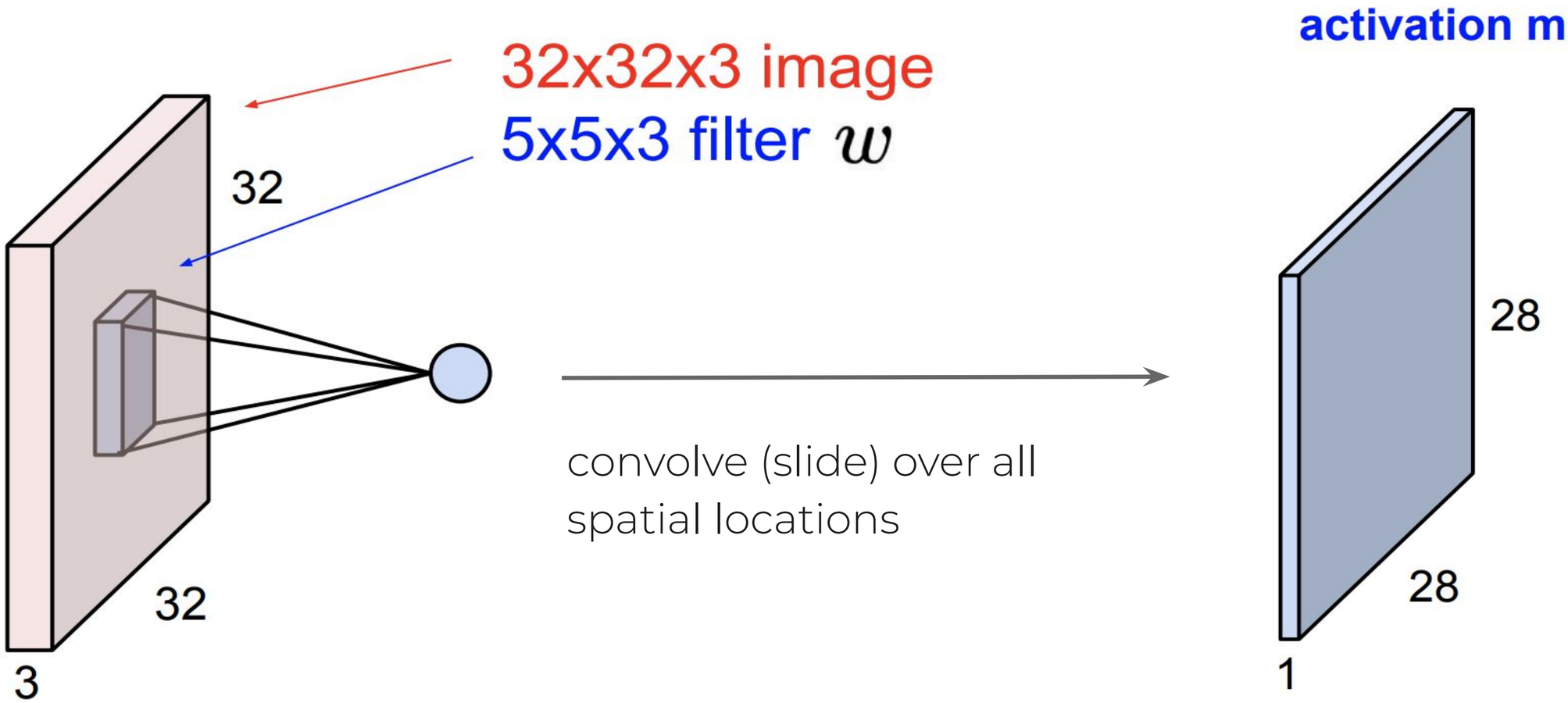
Image convolution



1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

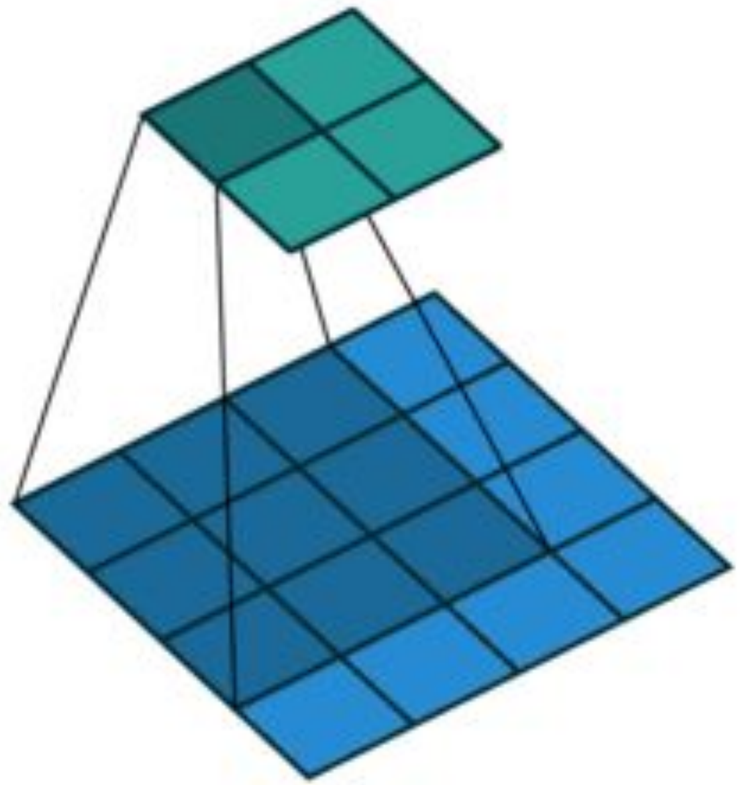
Image convolution



Convolutional layer

To make convolution be a neural network part we just need to adjust filter (aka kernel) weights with gradients (which we compute by backpropagation).

Since convolution is a differentiable operator everything works fine.

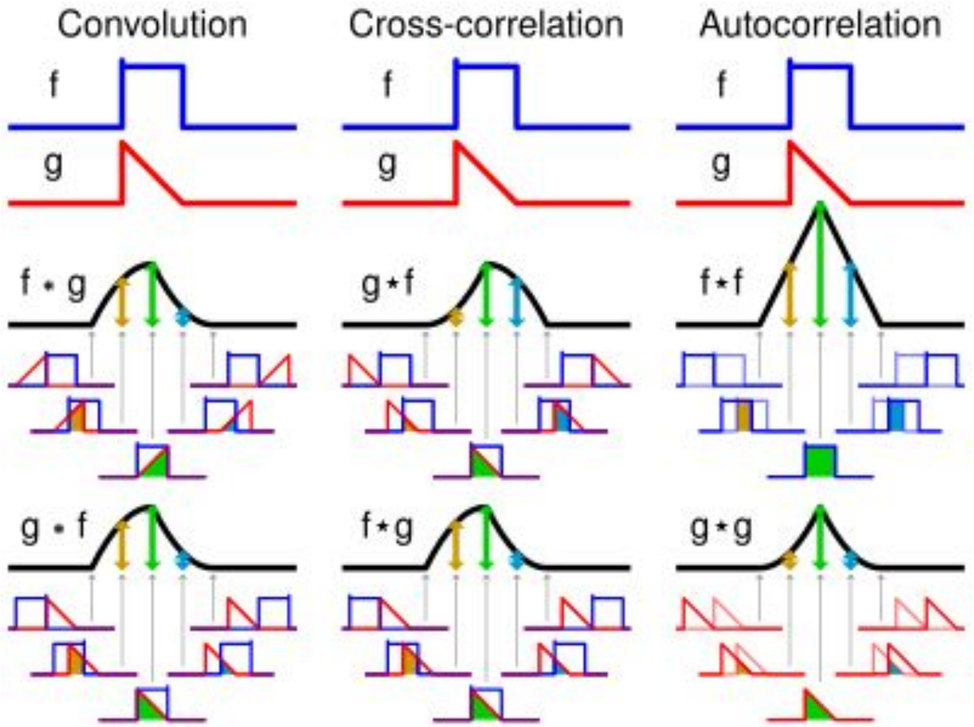


Cross-correlation vs Convolution

Formally speaking we use cross-correlation instead of convolution (because output is not mirrored).

But common term in ML for today is convolution.

Clarification: https://dsp.stackexchange.com/questions/2654/what-is-the-difference-between-convolution-and-cross-correlation/



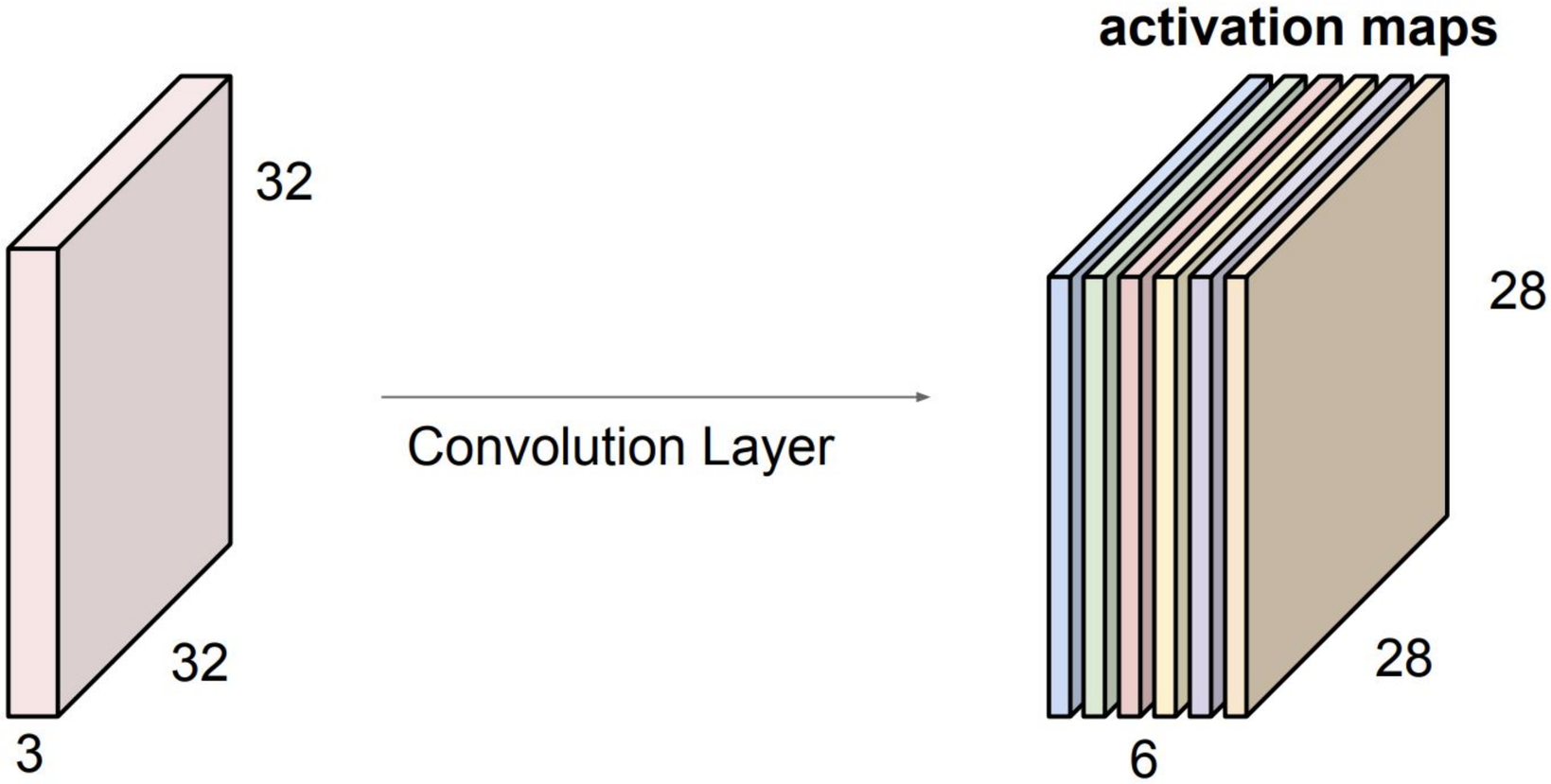
Conv layer hyperparams



- out channels
- kernel size
- stride
- padding
- dilation
- groups

Number of filters

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a "new image" of size 28x28x6!

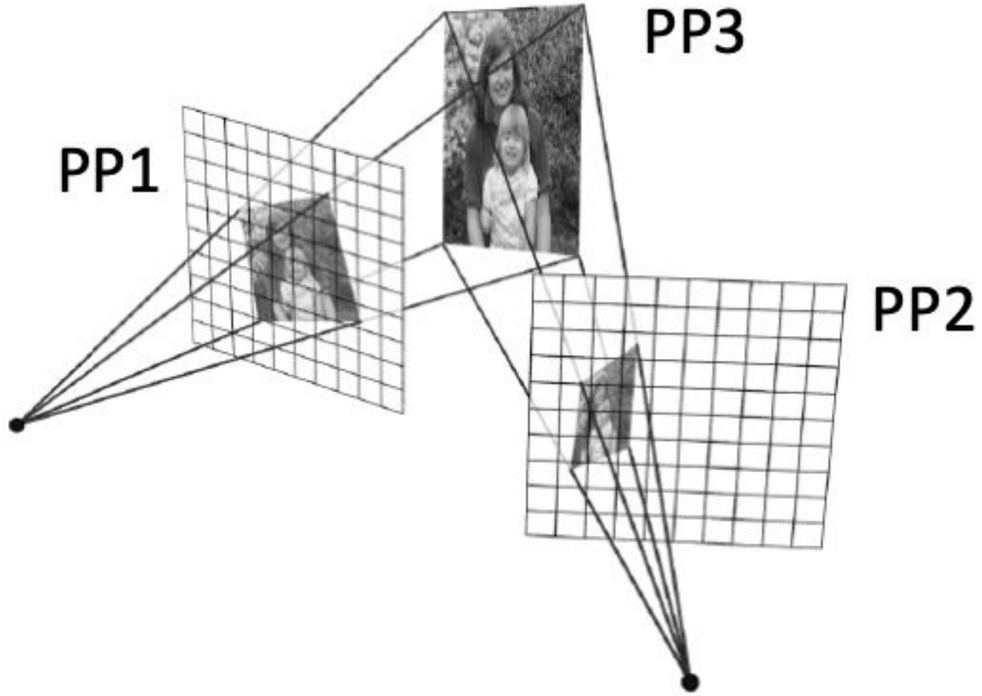
Invariant transforms for images

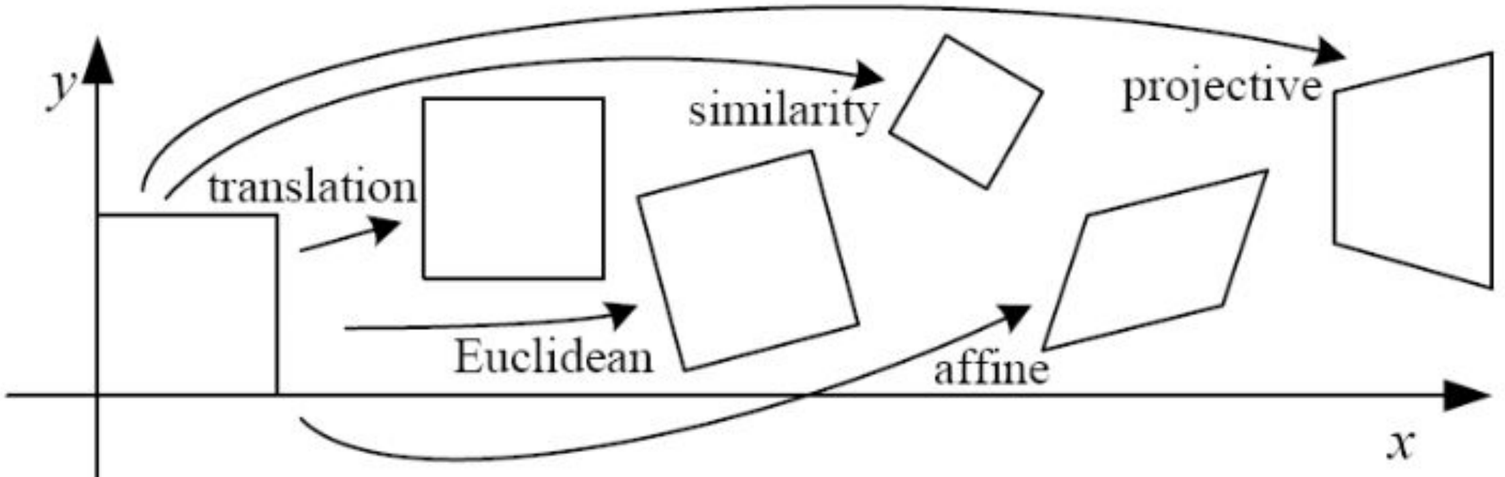
- Transition
- Scaling
  - Mirroring
- Rotation



Invariant transforms for images

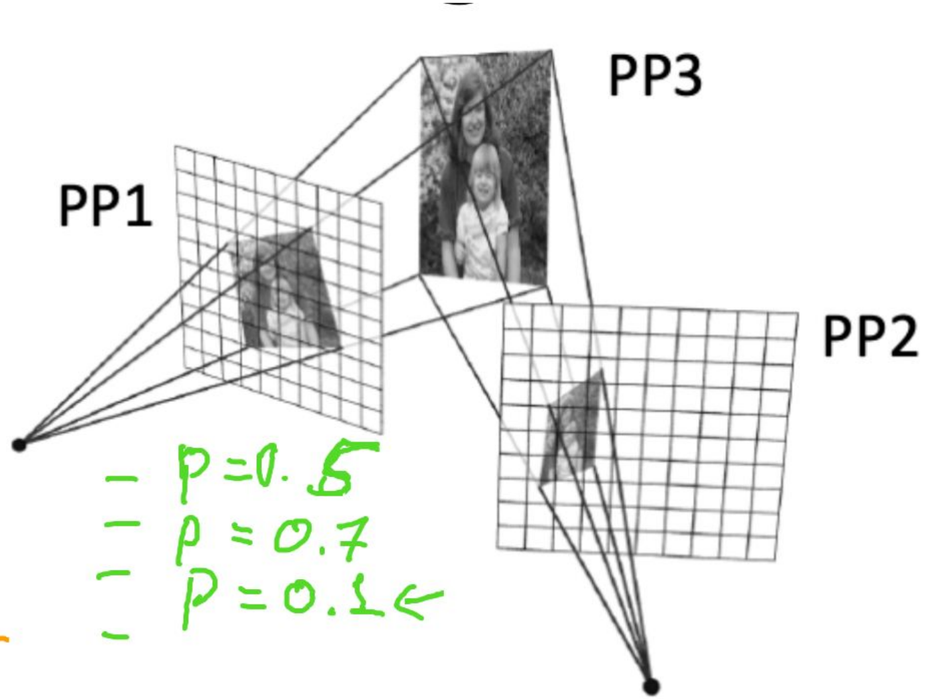
- Transition +
- Scaling +-
  - Mirroring +
- Rotation +-
- Homography +-
- Blur +-
- Luminance level -
- Colour noise -

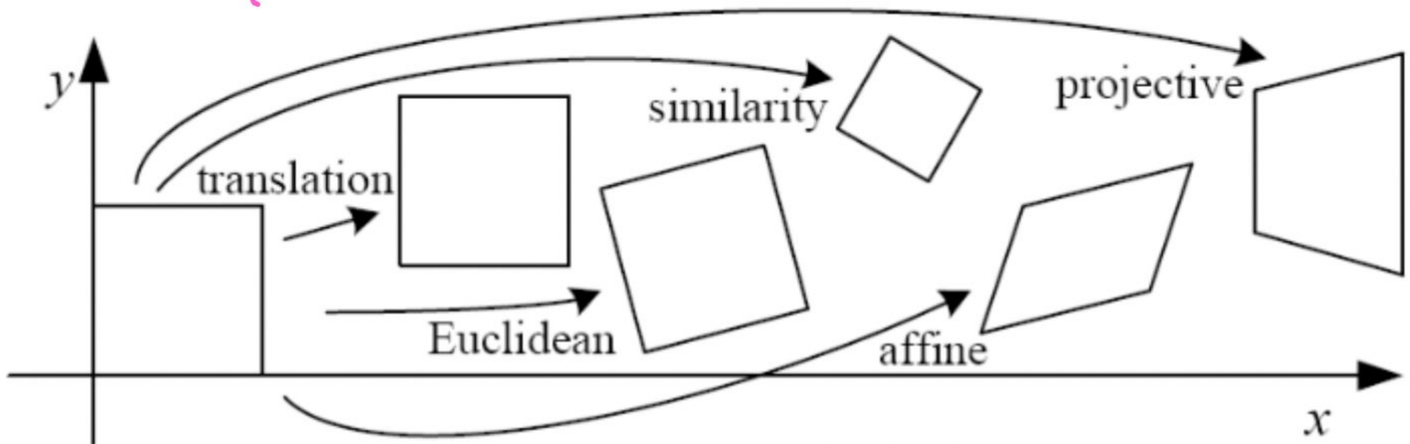




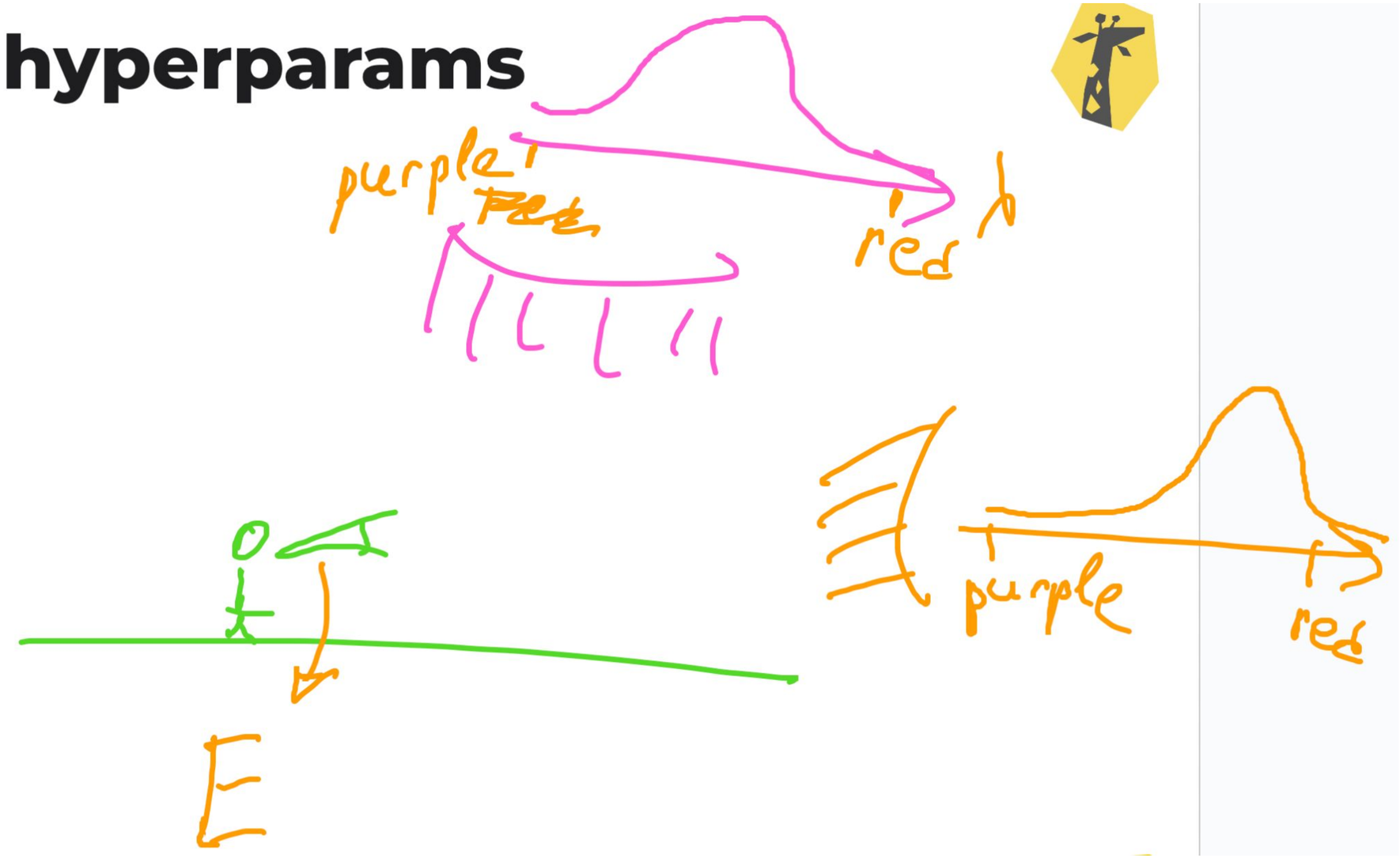
Invariant transforms for images

- Transition +
- Scaling +
  - Mirroring +
- Rotation +-
- Homography +-
- Blur +-
- Luminance level -
- Colour noise -

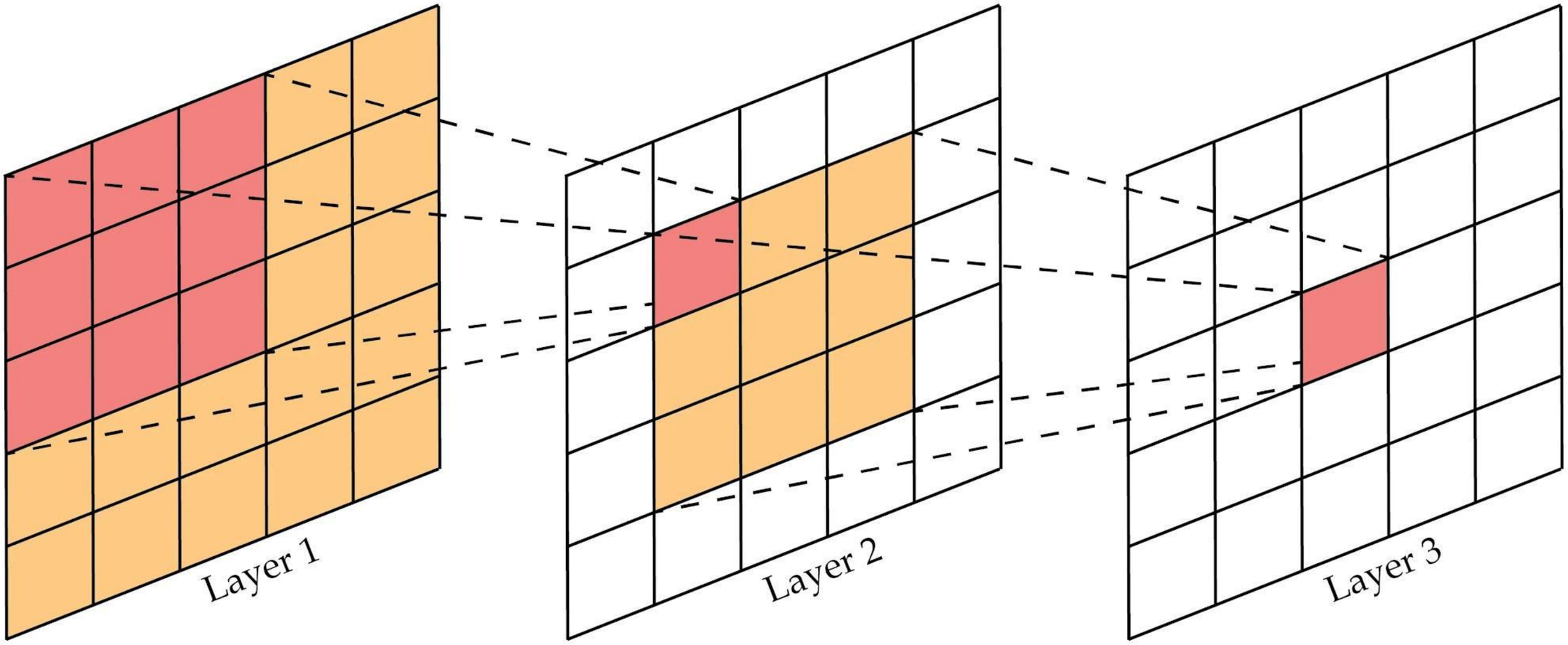




Invariant transforms for images



Receptive field



Kernel sizes

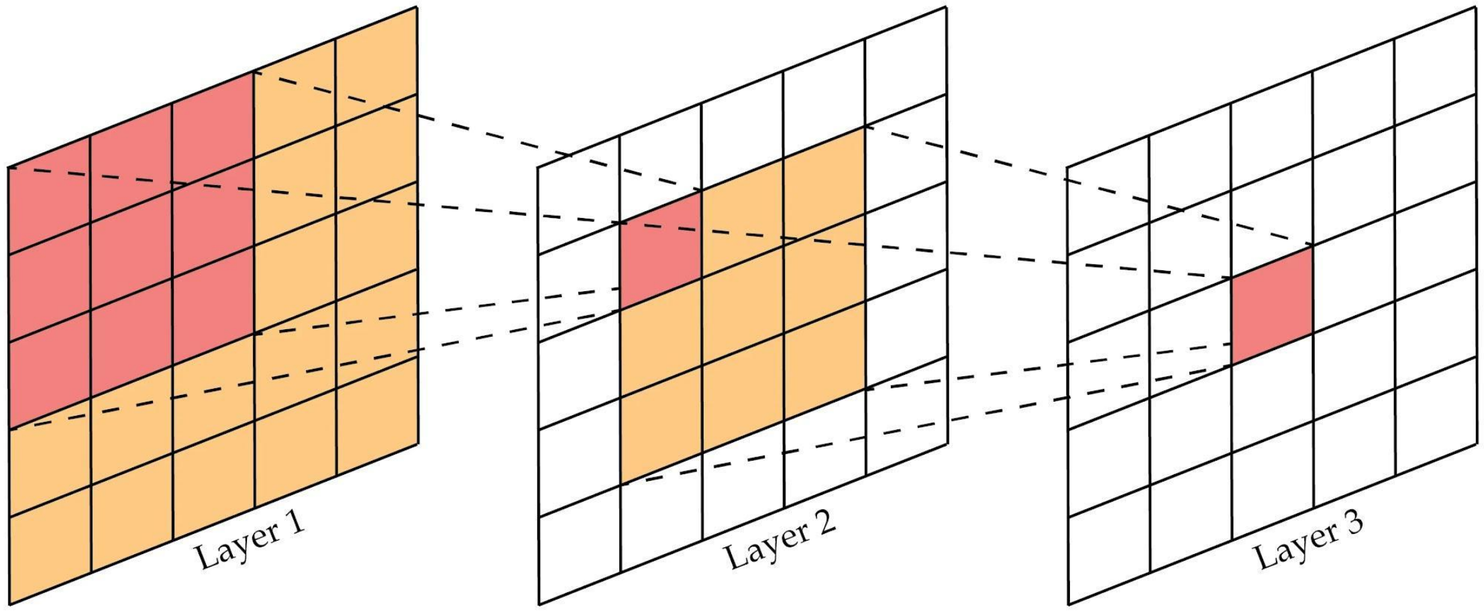


- even - not used in practice
  - 2
  - 4
  - 6
- odd - only such kernels are used
  - 1
  - 3
  - 5

Why?

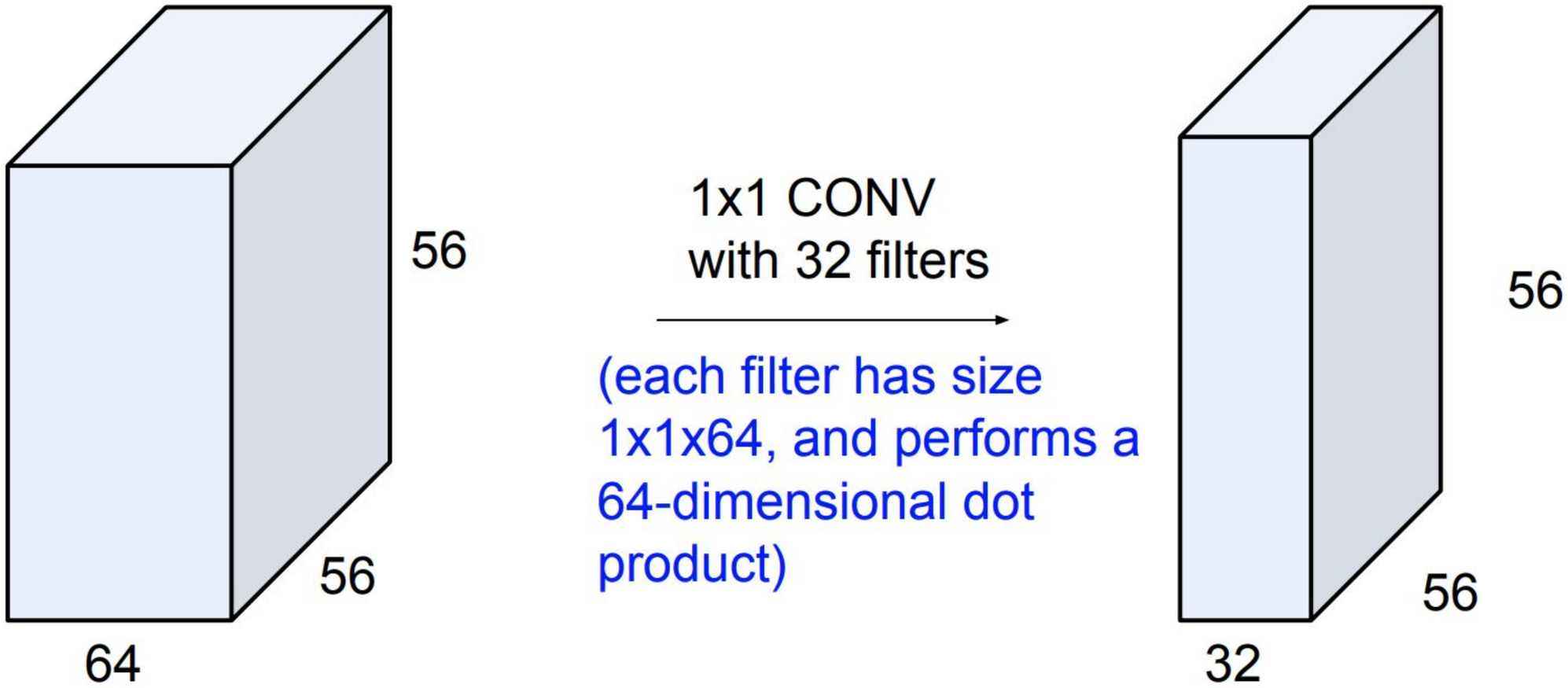
Comparing kernel sizes

1. Single 3x3 convolution
   a. Receptive Field - 3
   b. Parameters - 10
2. Single 5x5 convolution
   a. Receptive Field - 5
   b. Parameters - 26
3. Double 3x3 convolution
   a. Receptive Field - 5
   b. Parameters - 20



Double 3x3 is better because of receptive field and number of parameters.
It introduces more nonlinearity to feature maps.
Although latency of two convolutions is higher.

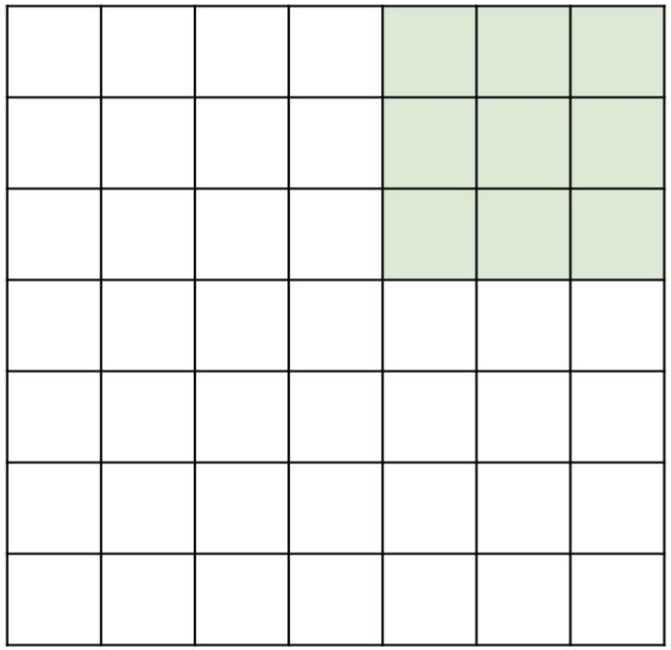
Pointwise (1x1) convolution



- changes number of channels with minimal cost
- generates new features without spatial component
- equivalent to linear layer over channels (spatially local linear layer)

Stride

A closer look at spatial dimensions:

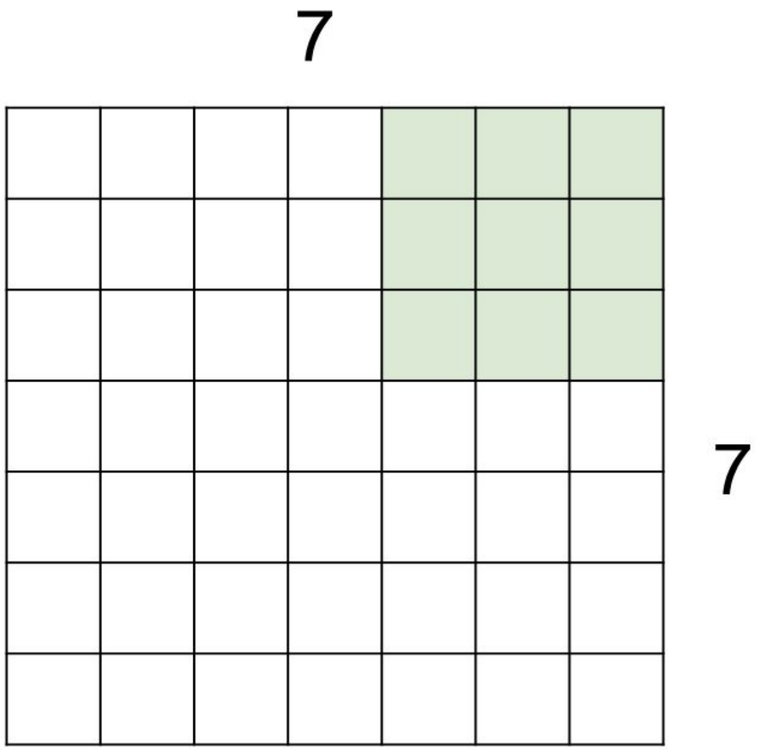


7x7 input (spatially)
assume 3x3 filter

=> 5x5 output

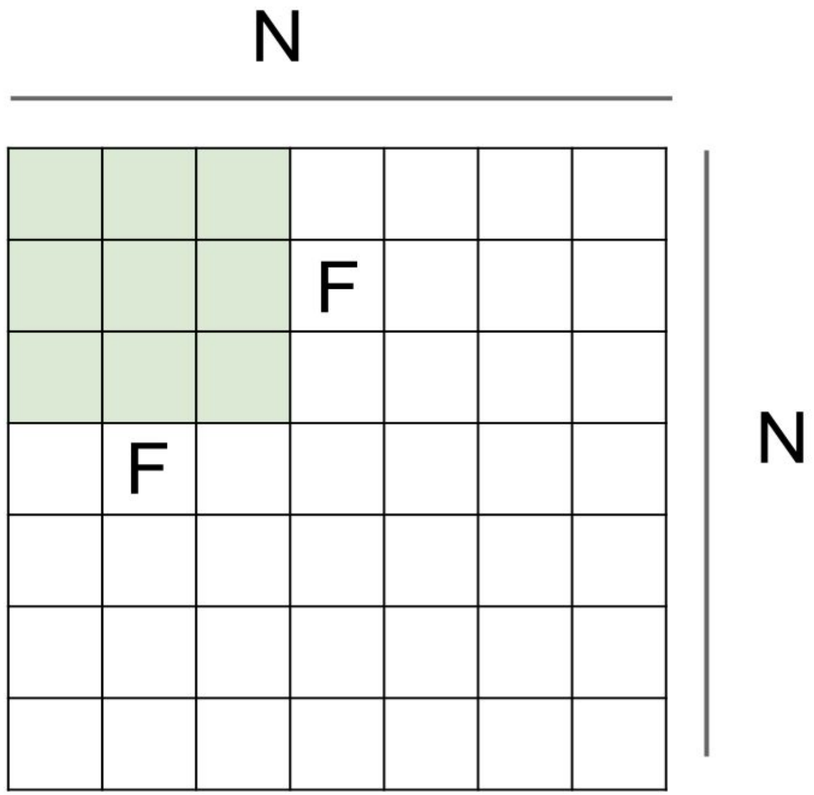
Stride

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

Stride output size



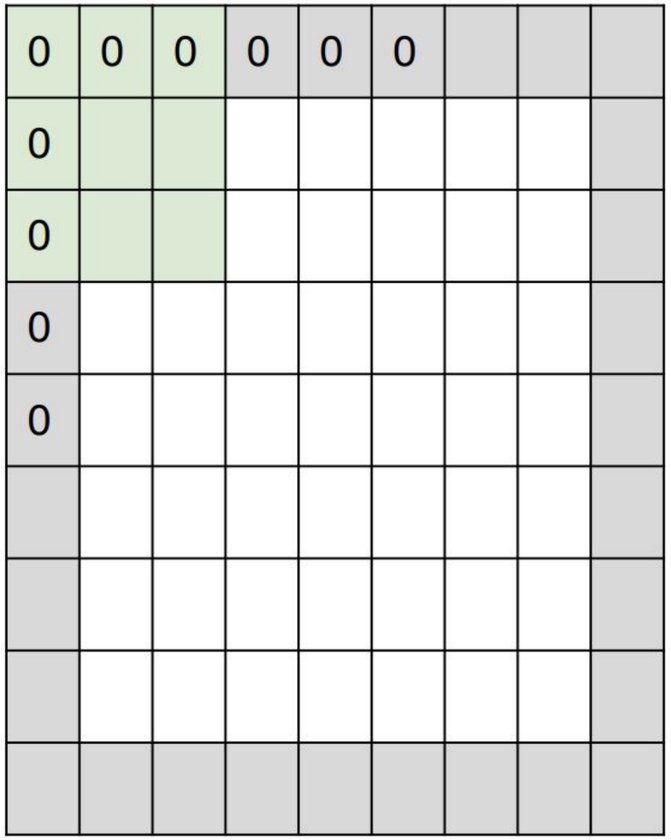
Output size:
(N - F) / stride + 1

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

Padding



In practice: Common to zero pad the border



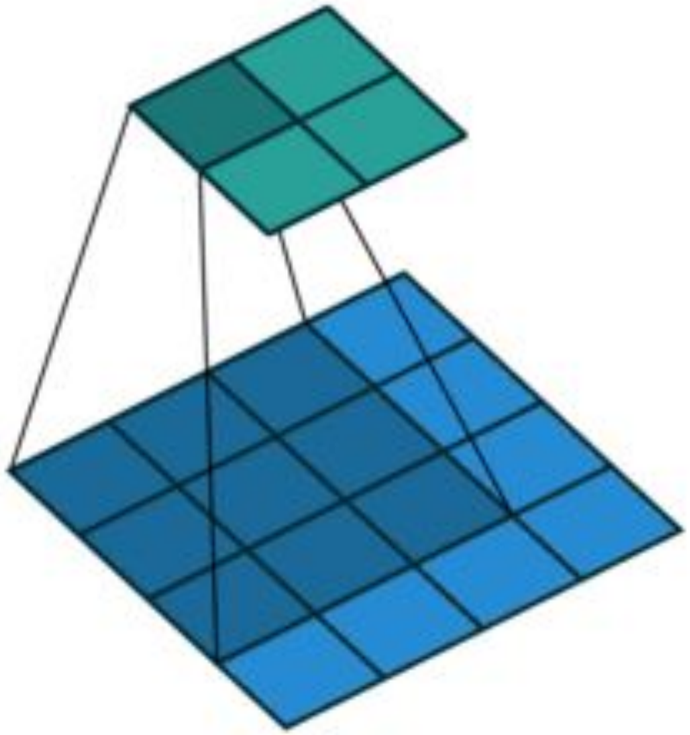
e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

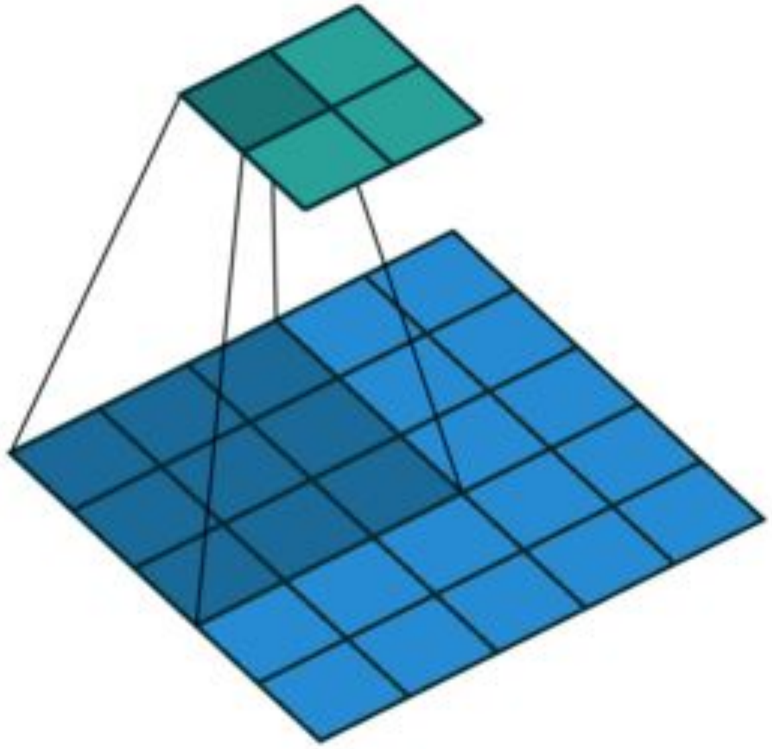
Padding mode

- zeros
- reflect
- replicate
- circular

Strides demo

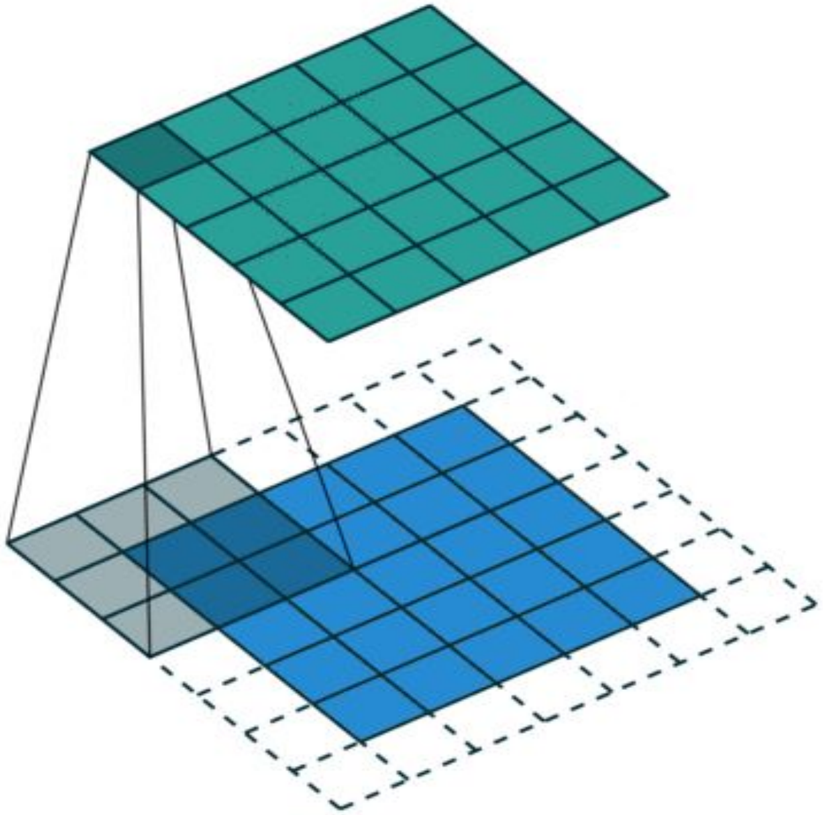


No padding, no strides

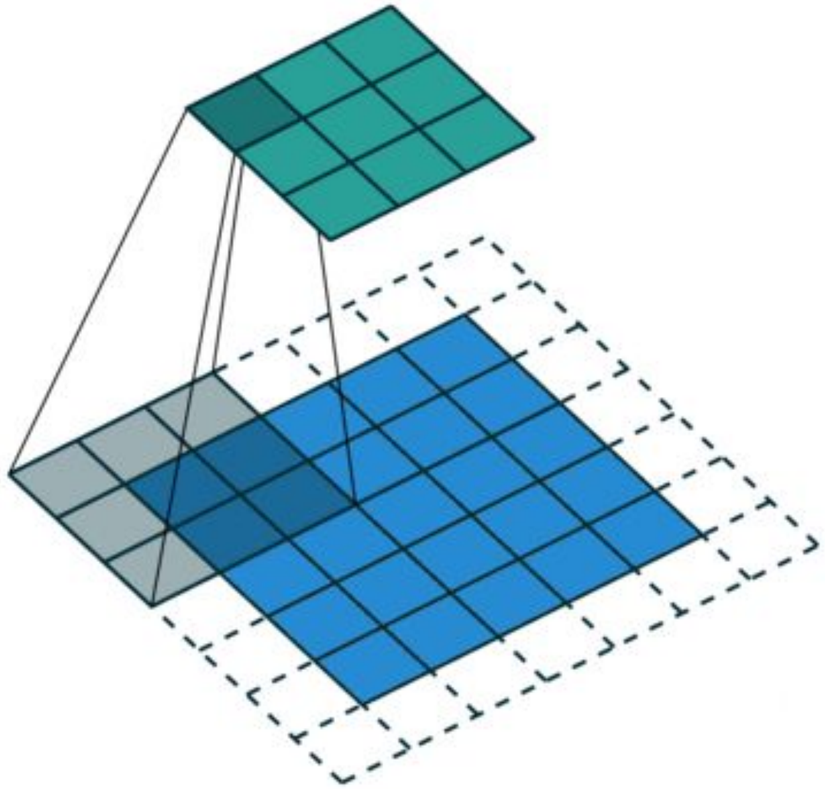


No padding, with strides

Padding and strides demo



With padding, no strides



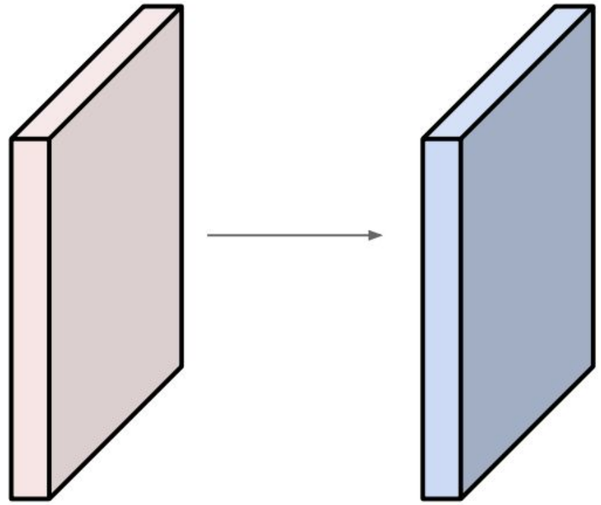
With padding, with strides

Output size calculation

Examples time:

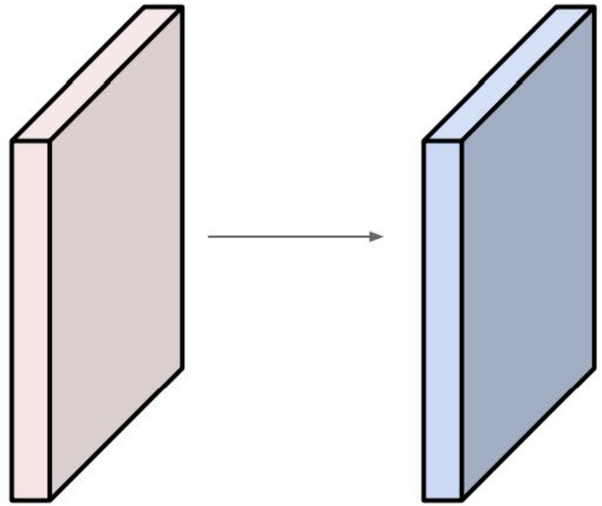
Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?



Output size calculation

Examples time:

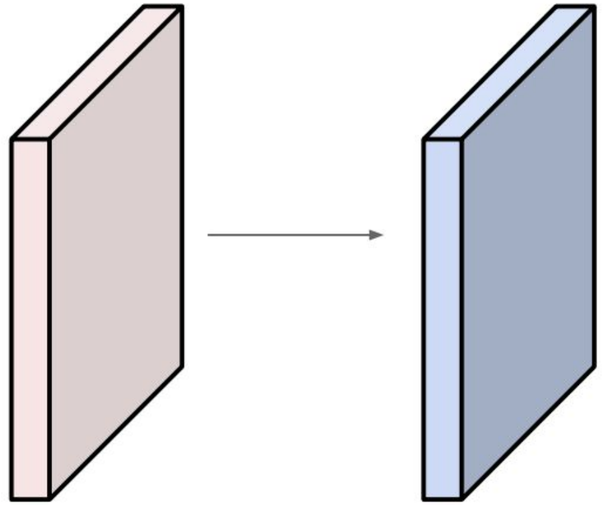


Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10

Parameters number calculation

Examples time:

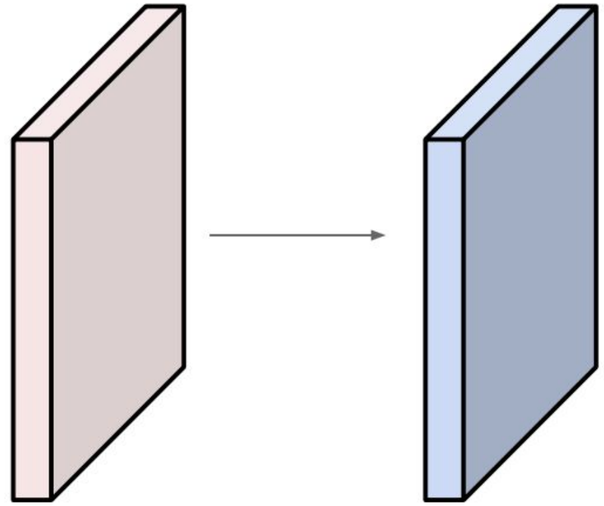


Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?

Parameters number calculation

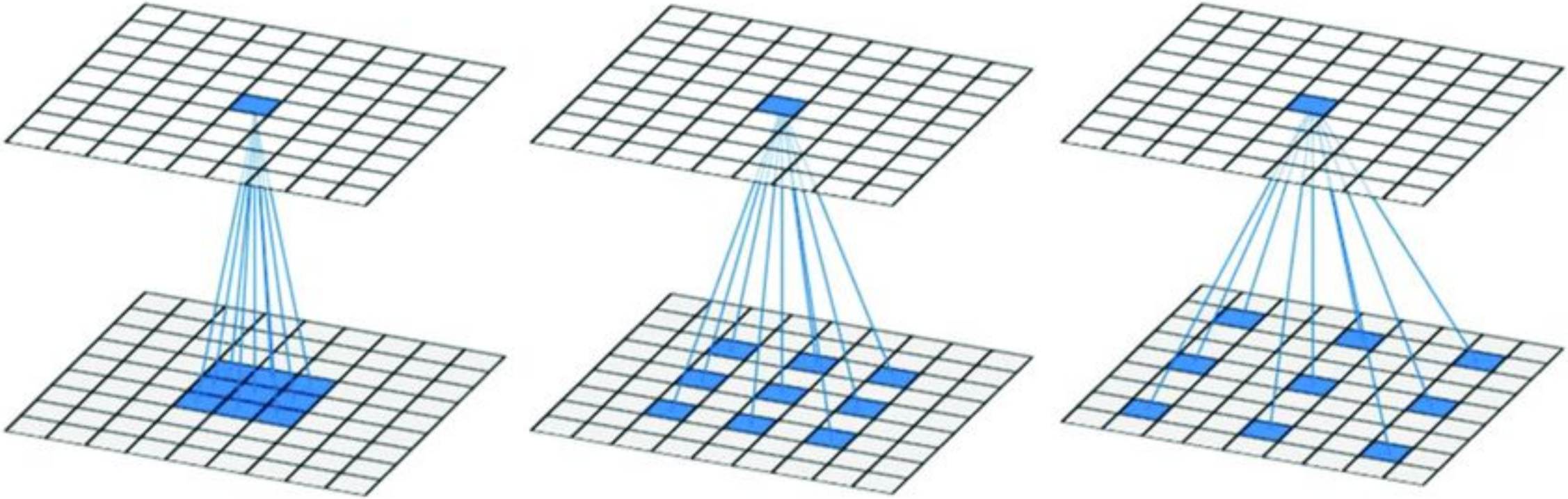
Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = **760**

Dilation



dilation=1 dilation=2 dilation=3

Spatial Separable Convolution



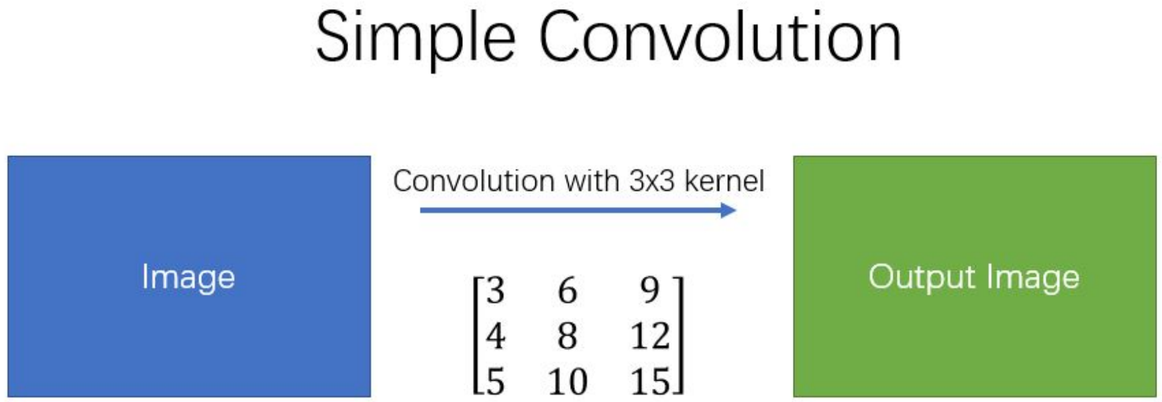
If we use two convolutions with vector kernels it is equivalent to some square kernel.

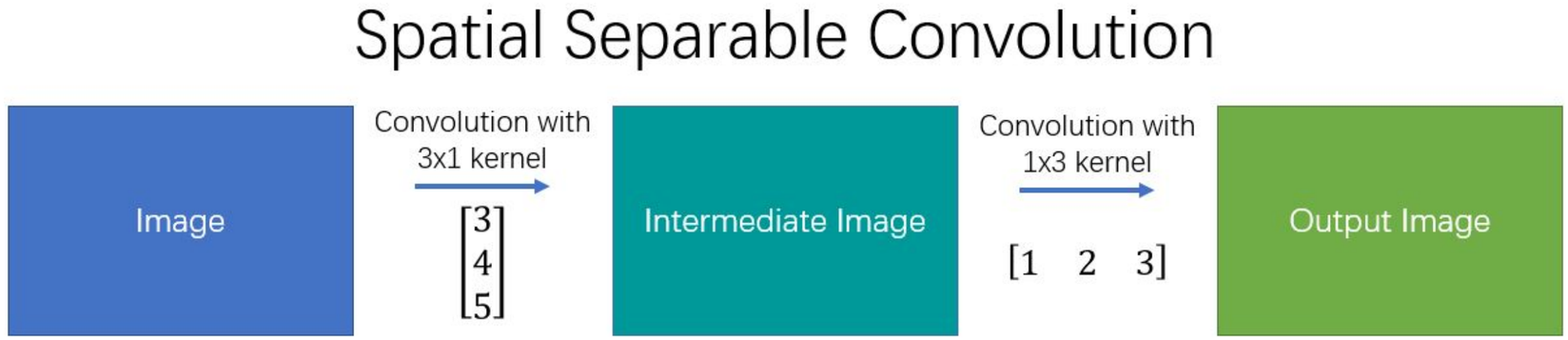
In this case number of parameters and flops is lower compared to square kernel.

Downside is that these are sequential operations, so latency is higher.

$$\begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix}$$

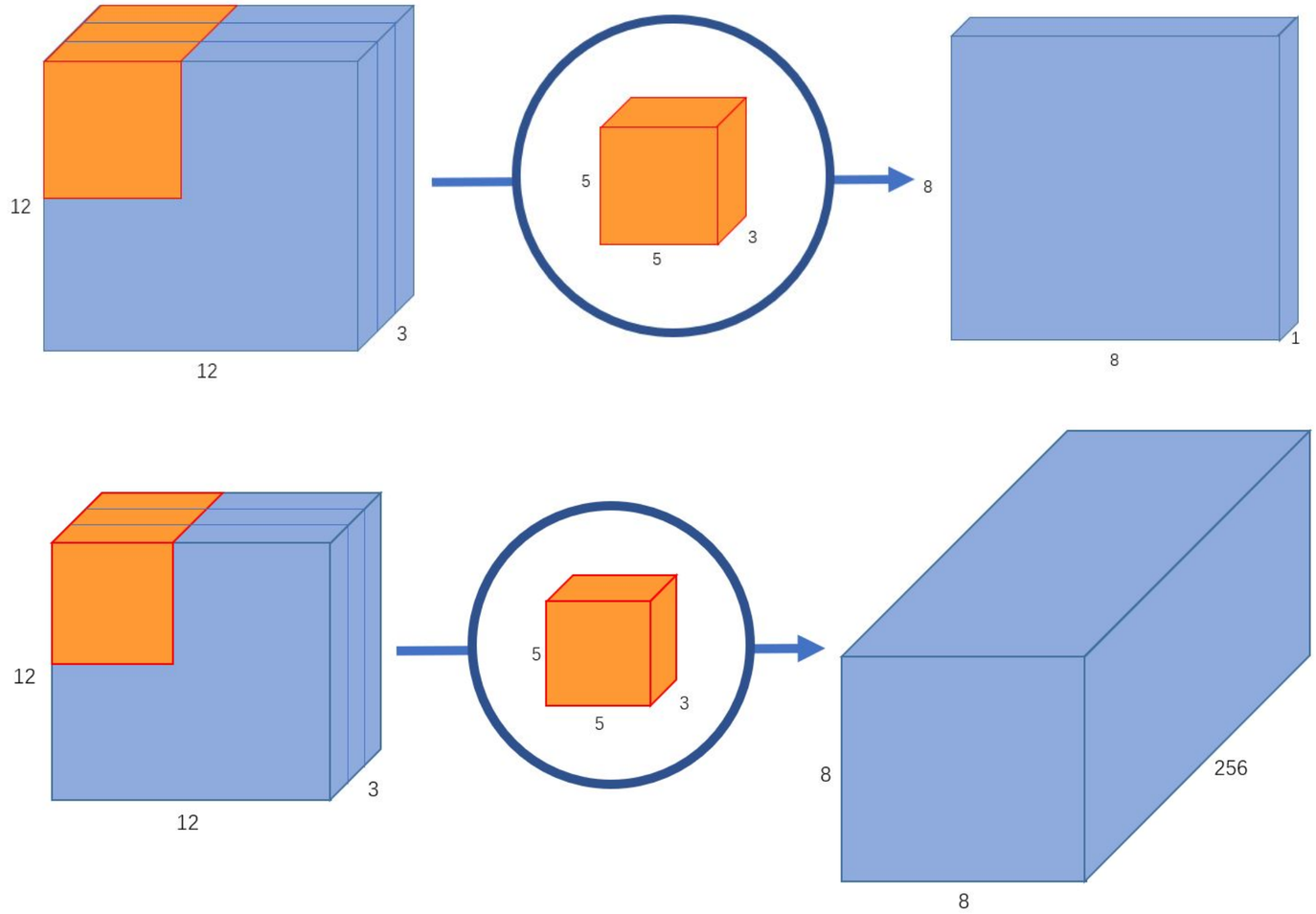
Spatial Separable Convolution



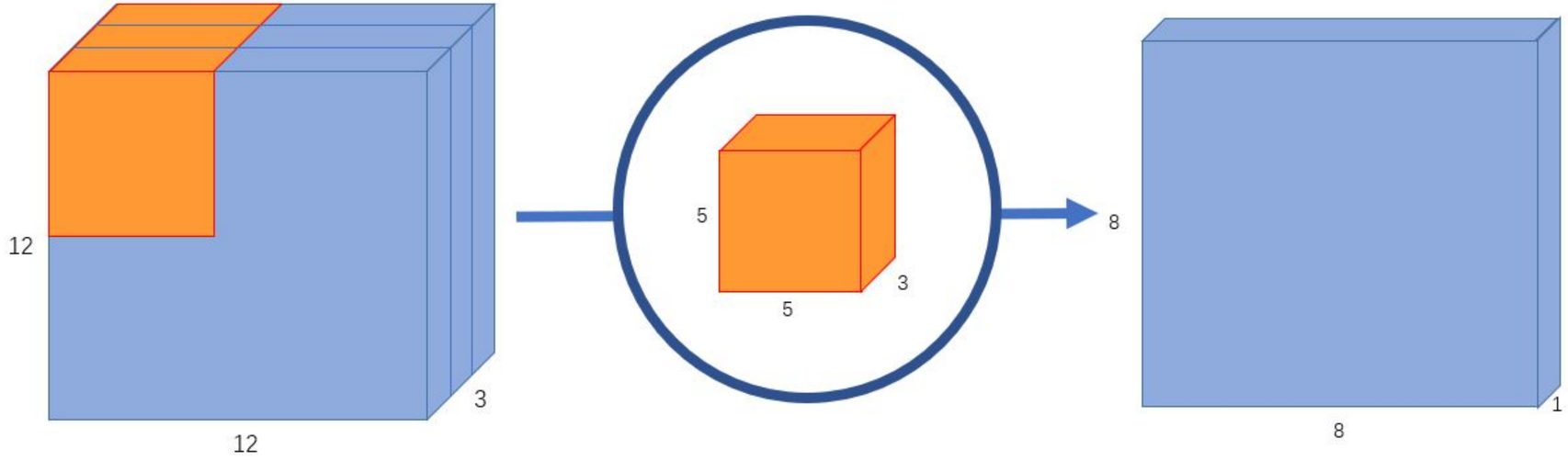


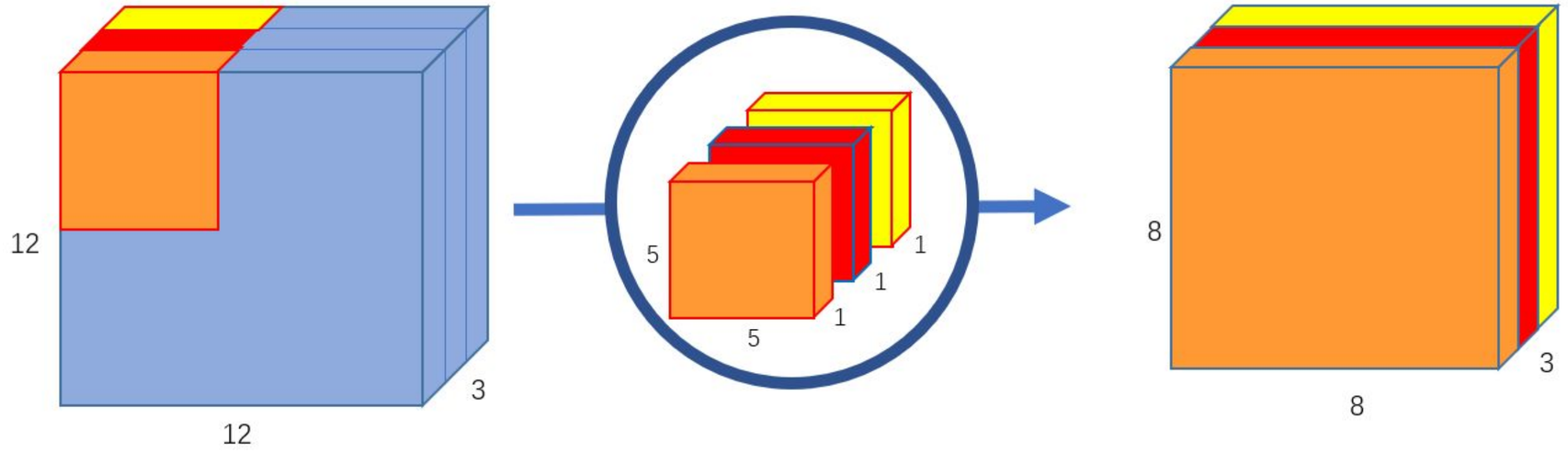
Just like with 5x5 vs 3x3 convs we exchange latency to

Normal Convolution



Depthwise Convolution





Conv layer hyperparams



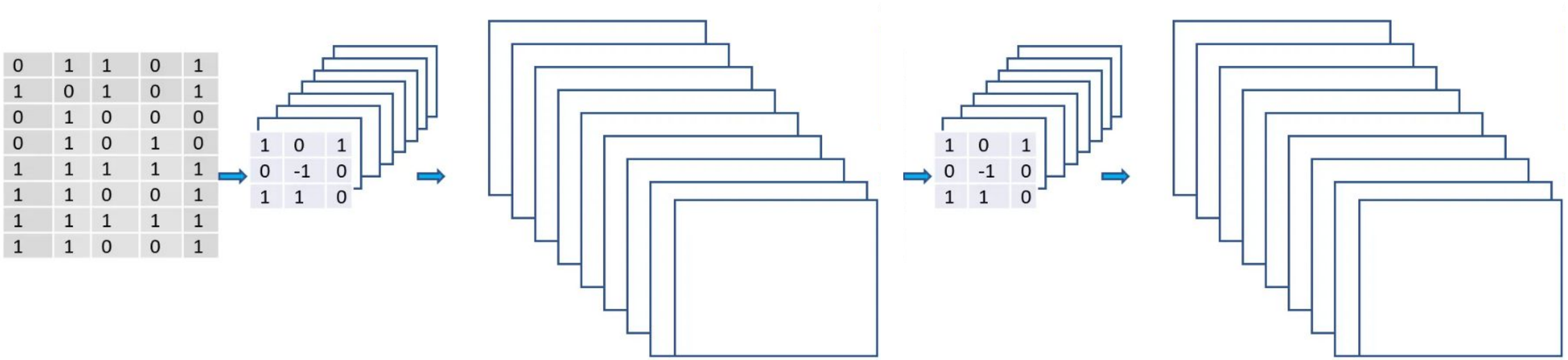
- **out channels** - number of filters
- **kernel size** - spatial size of each filter
- **stride** - step of kernel
- **padding** - extend edges with numbers
- **dilation** - make kernel sparse
- **groups** - make depthwise convs

torch.nn.Conv2d

Memory consumption

For convolutional layers it’s big!

Because we have to store intermediate feature maps for all the stages.



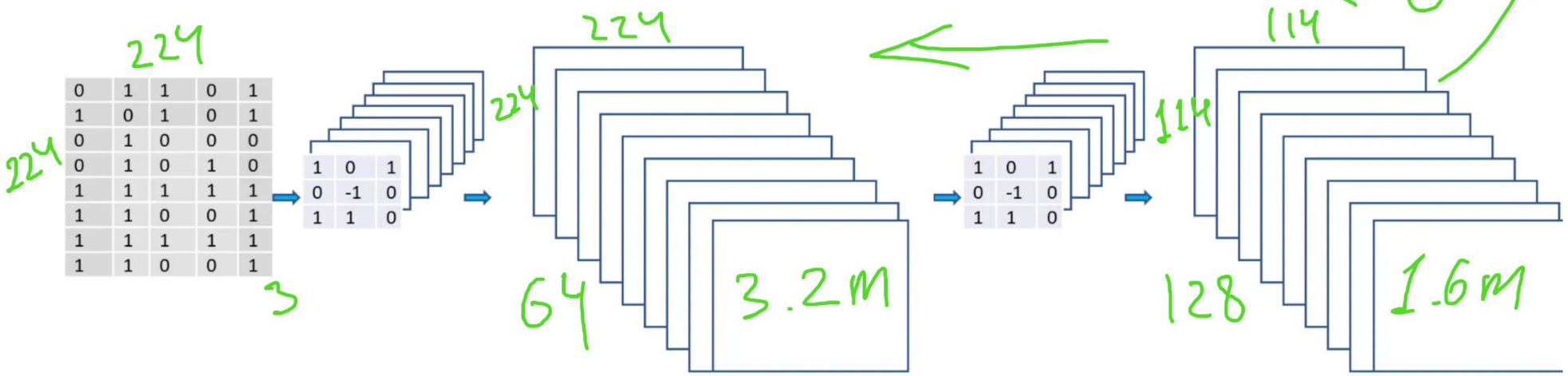
Input image Filters Intermediate featuremap More filters More featuremaps

Memory consumption

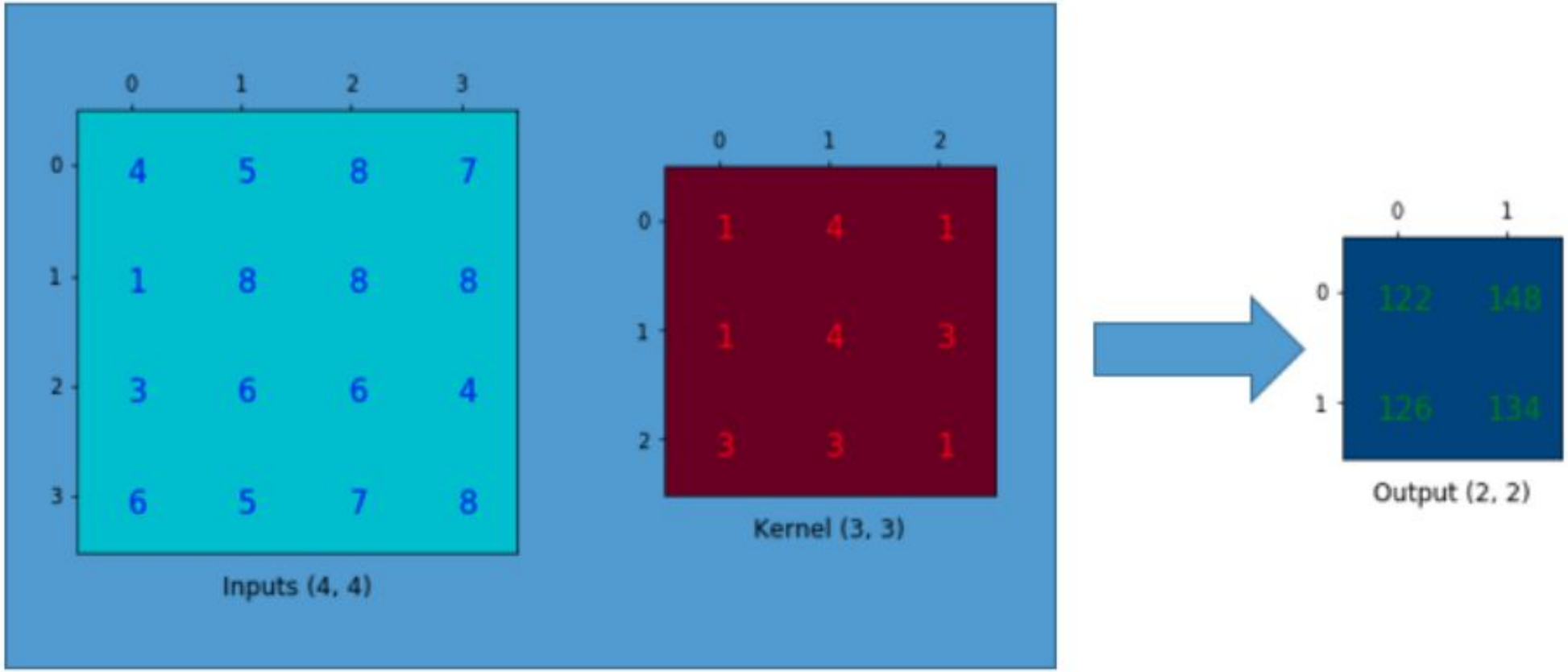
Memory consumption

For convolutional layers it's big!

Because we have to store intermediate feature maps for all the stages.

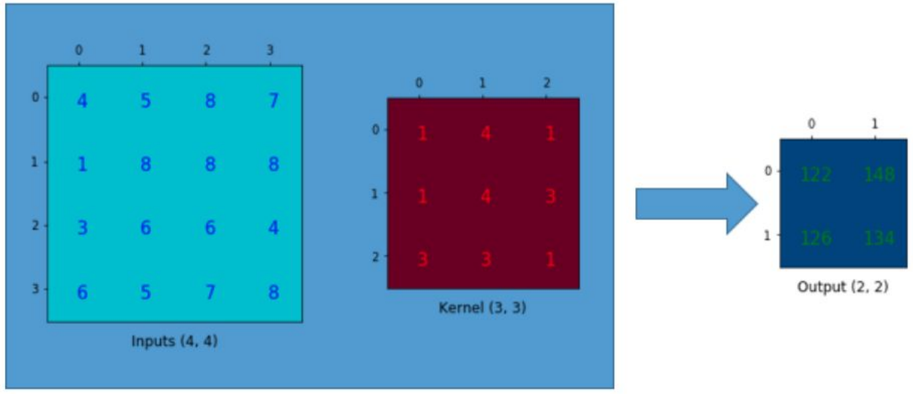


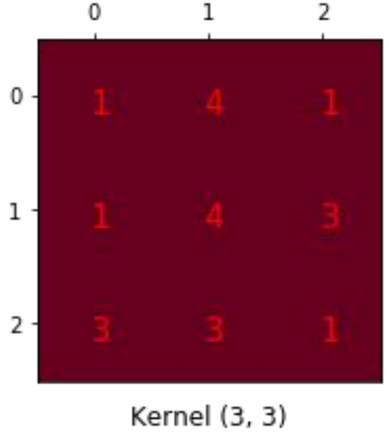
Convolution under the hood

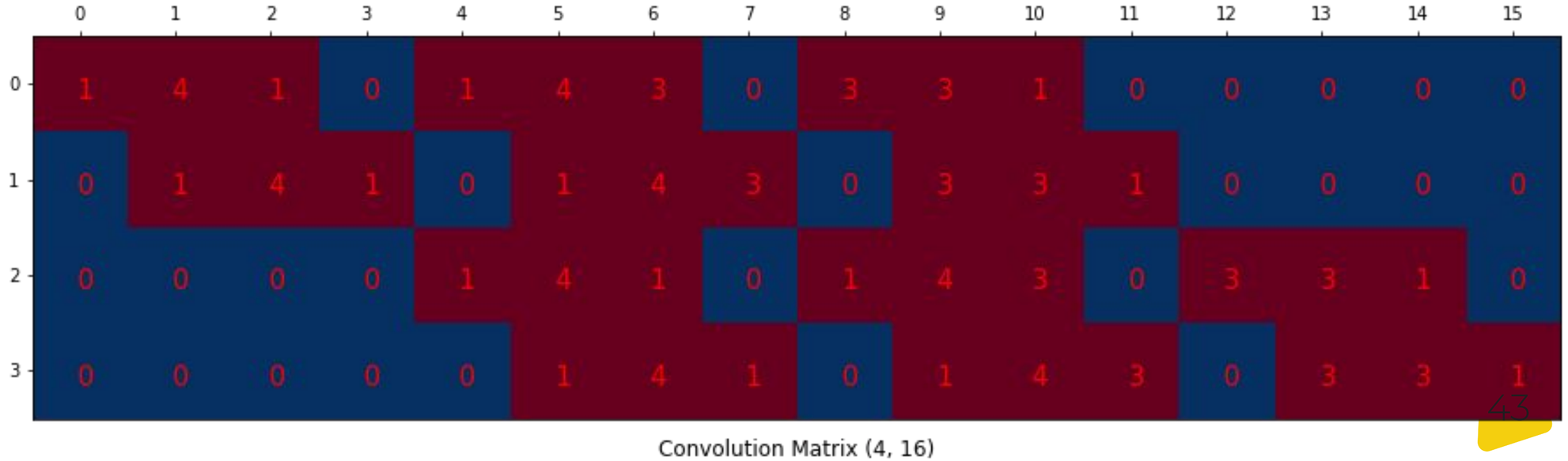


[Images credentials]

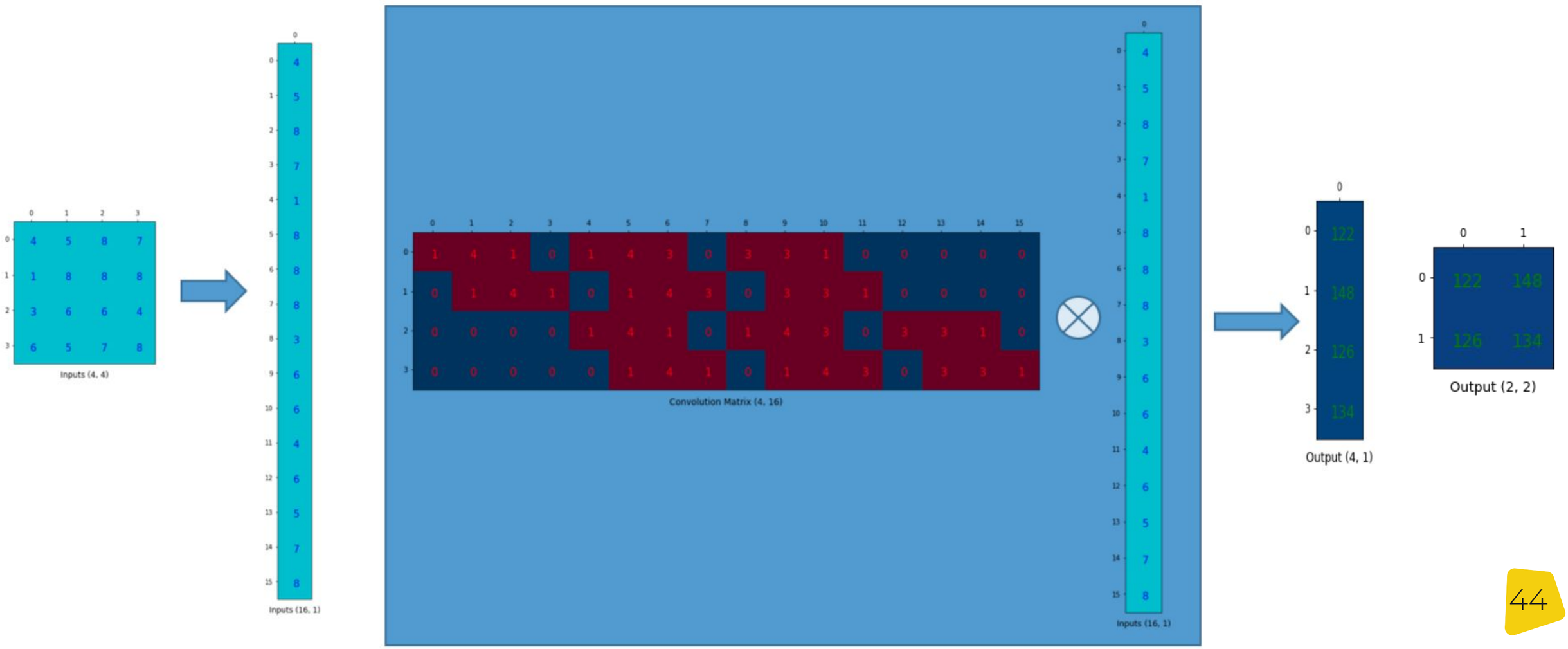
Convolutional matrix



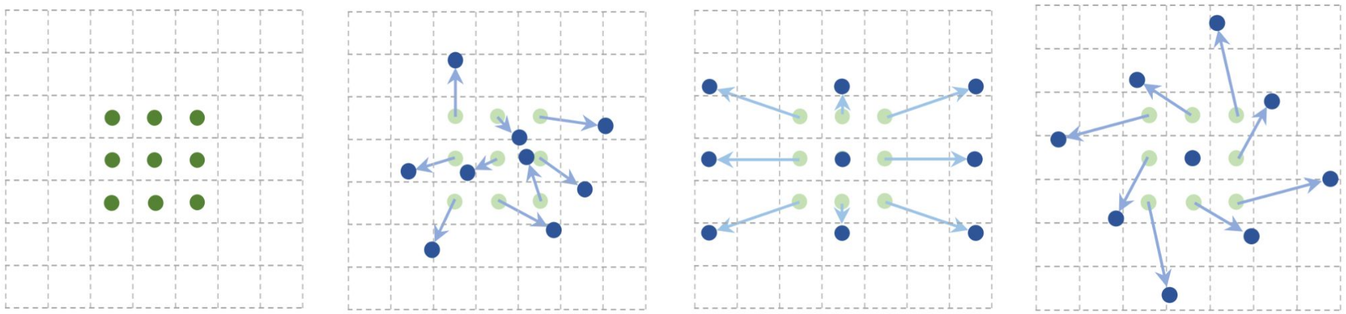


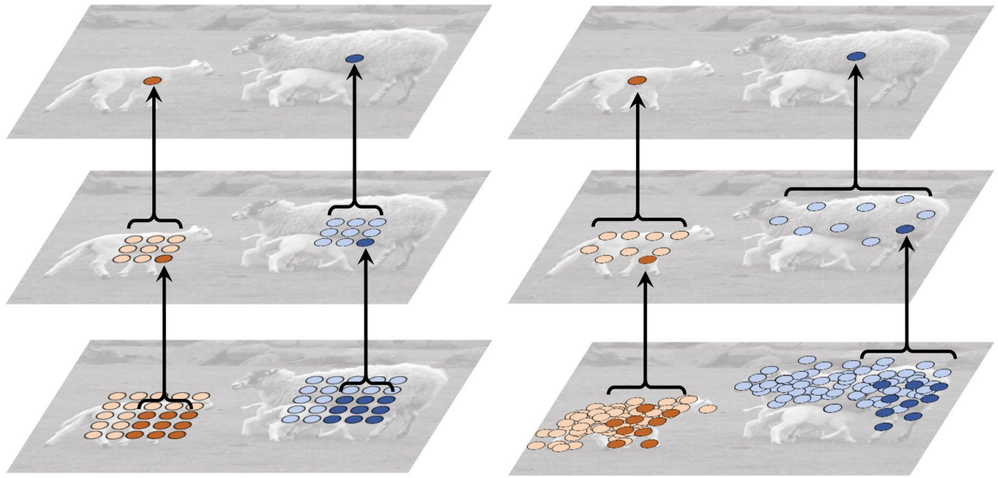


Convolution matrix form



Deformable convolutions





(a) standard convolution (b) deformable convolution

- https://arxiv.org/pdf/1703.06211
- https://arxiv.org/pdf/1811.11168
- https://docs.pytorch.org/vision/main/generated/torchvision.ops.deform_conv2d.html
- https://www.youtube.com/watch?v=HRLMSrxw2To
- https://www.youtube.com/watch?v=6TtOuVJ9GBQ

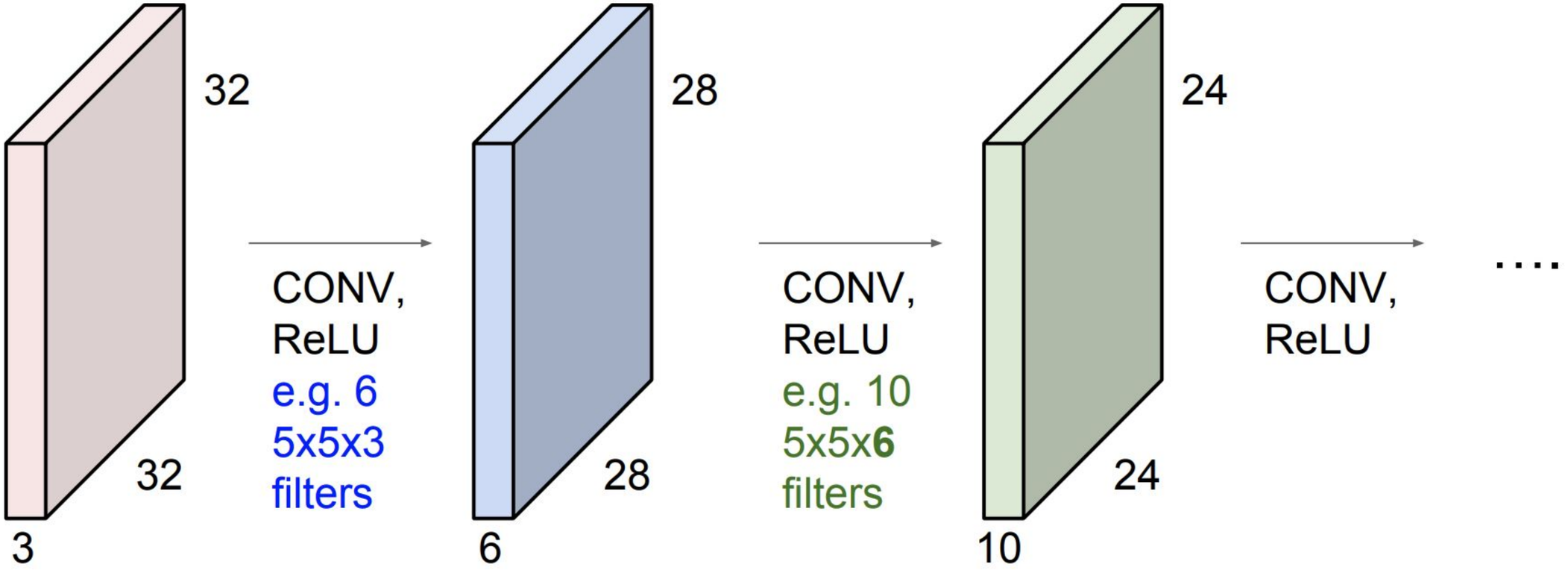
CNN filter analysis

girafe ai

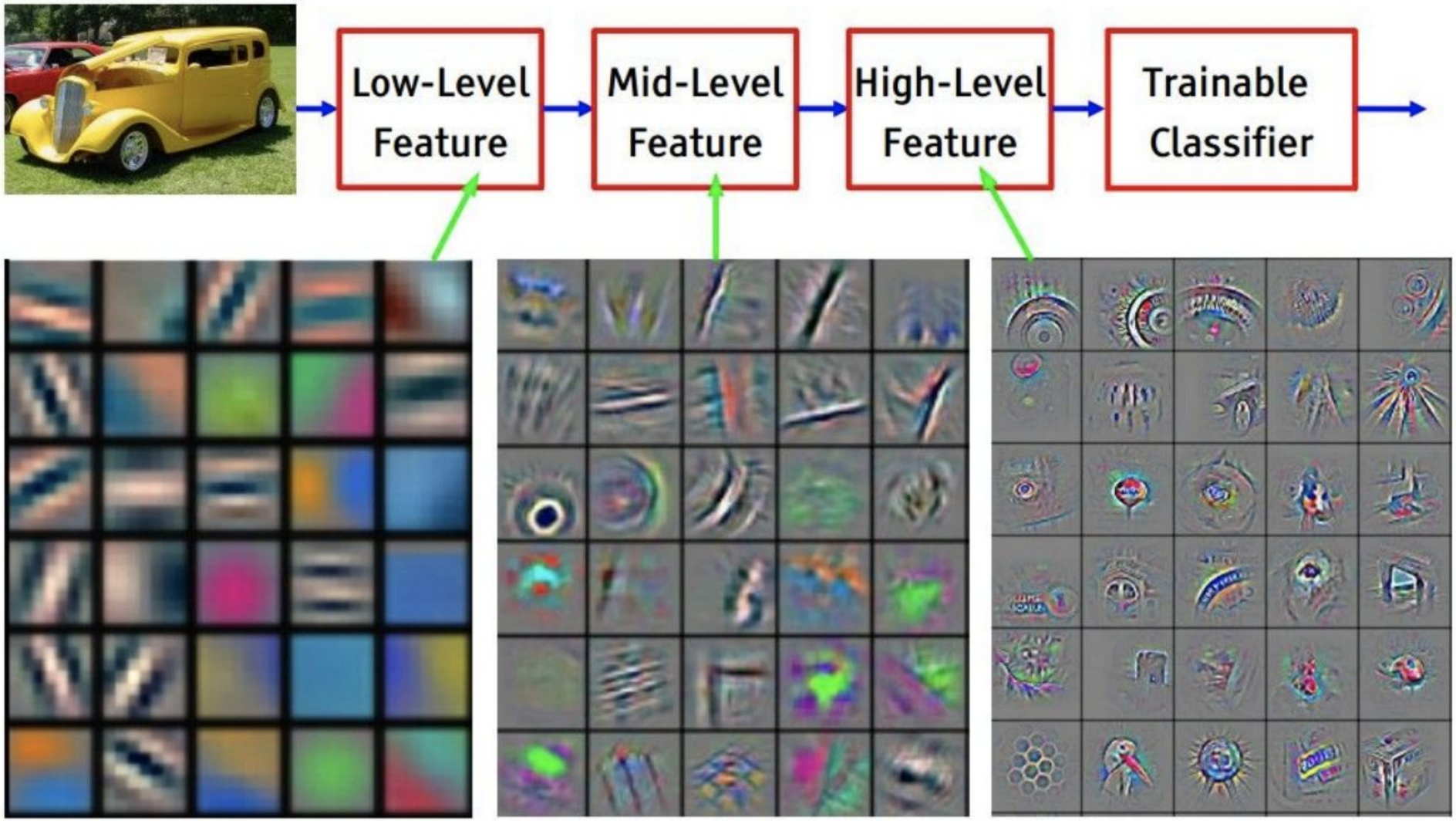
02

Convolutional Neural Network

Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



Trained convolution kernels



Visualizing and Understanding Convolutional Networks, Zeiler, Fergus, 2013

DeepDream



DeepDream links



- https://research.google/blog/deepdream-a-code-example-for-visualizing-neural-networks/
- https://en.wikipedia.org/wiki/DeepDream
- https://research.google/blog/inceptionism-going-deeper-into-neural-networks/
- https://www.youtube.com/watch?v=DgPaCWJL7XI

CNN artifacts

- ImageNet-trained CNNs are biased towards texture, 2018
  - https://arxiv.org/abs/1811.12231



(a) Texture image

81.4%	**Indian elephant**
10.3%	indri
8.2%	black swan



(b) Content image

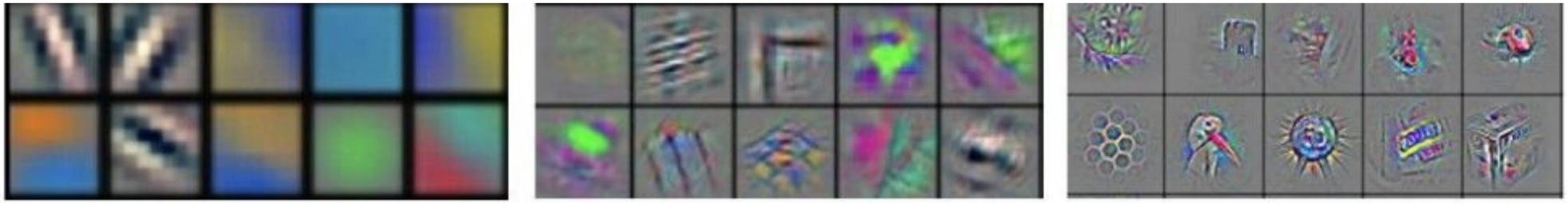
71.1%	**tabby cat**
17.3%	grey fox
3.3%	Siamese cat



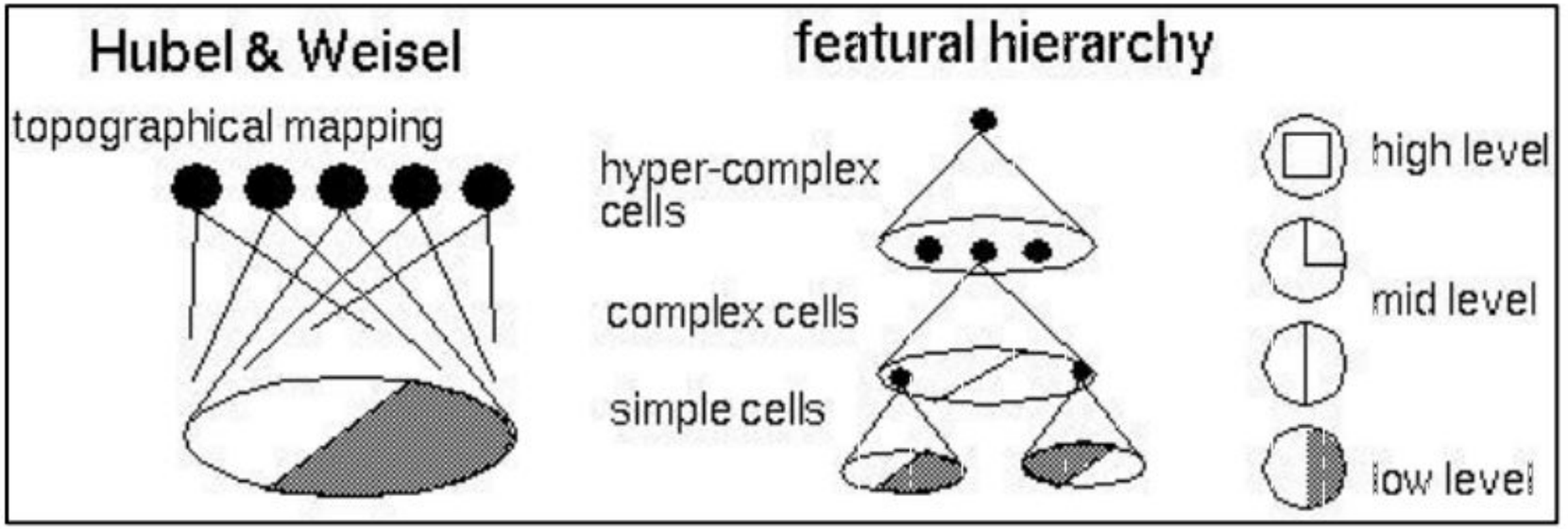
(c) Texture-shape cue conflict

63.9%	**Indian elephant**
26.4%	indri
9.6%	black swan

Convolutional layer and visual cortex



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

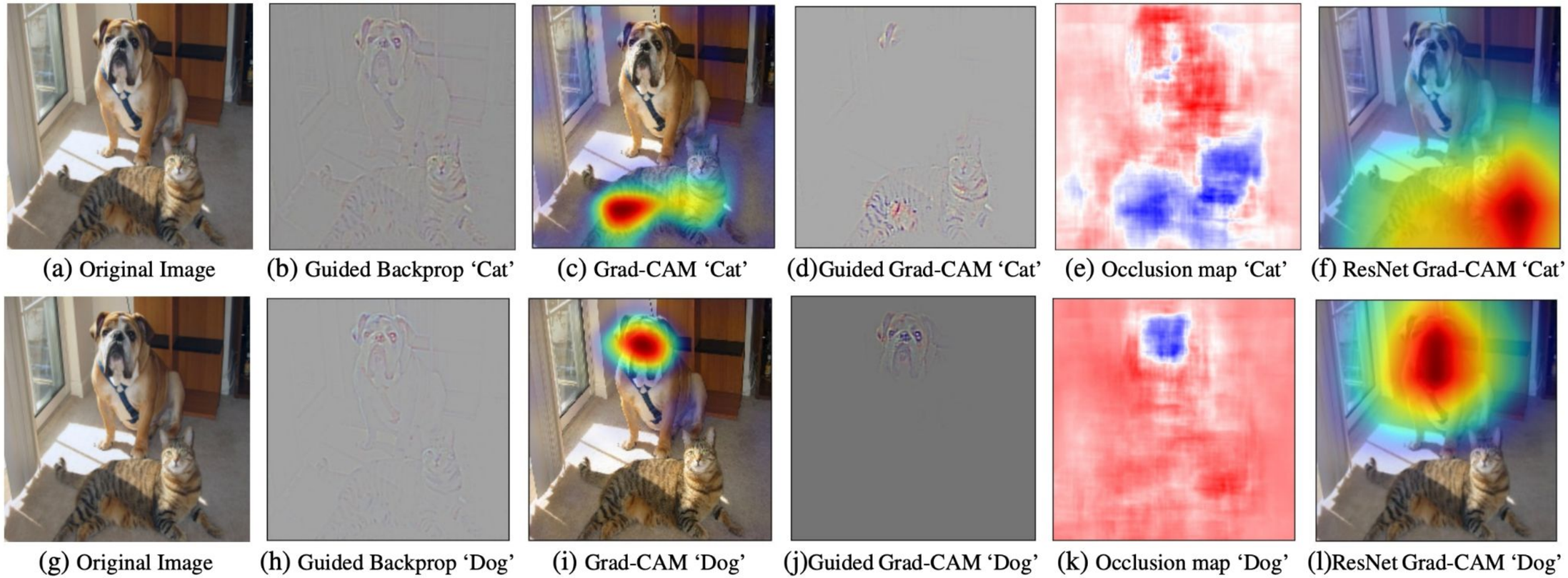


Convolutional layer and visual cortex



Gradients meaning

- Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization https://arxiv.org/pdf/1610.02391
  - https://www.youtube.com/watch?v=nBqdUAYxLjs
- VISTA: A Panoramic View of Neural Representations



Adversarial attacks

- Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization https://arxiv.org/pdf/1610.02391
  - https://www.youtube.com/watch?v=nBqdUAYxLjs
- VISTA: A Panoramic View of Neural Representations https://arxiv.org/html/2412.02412v1

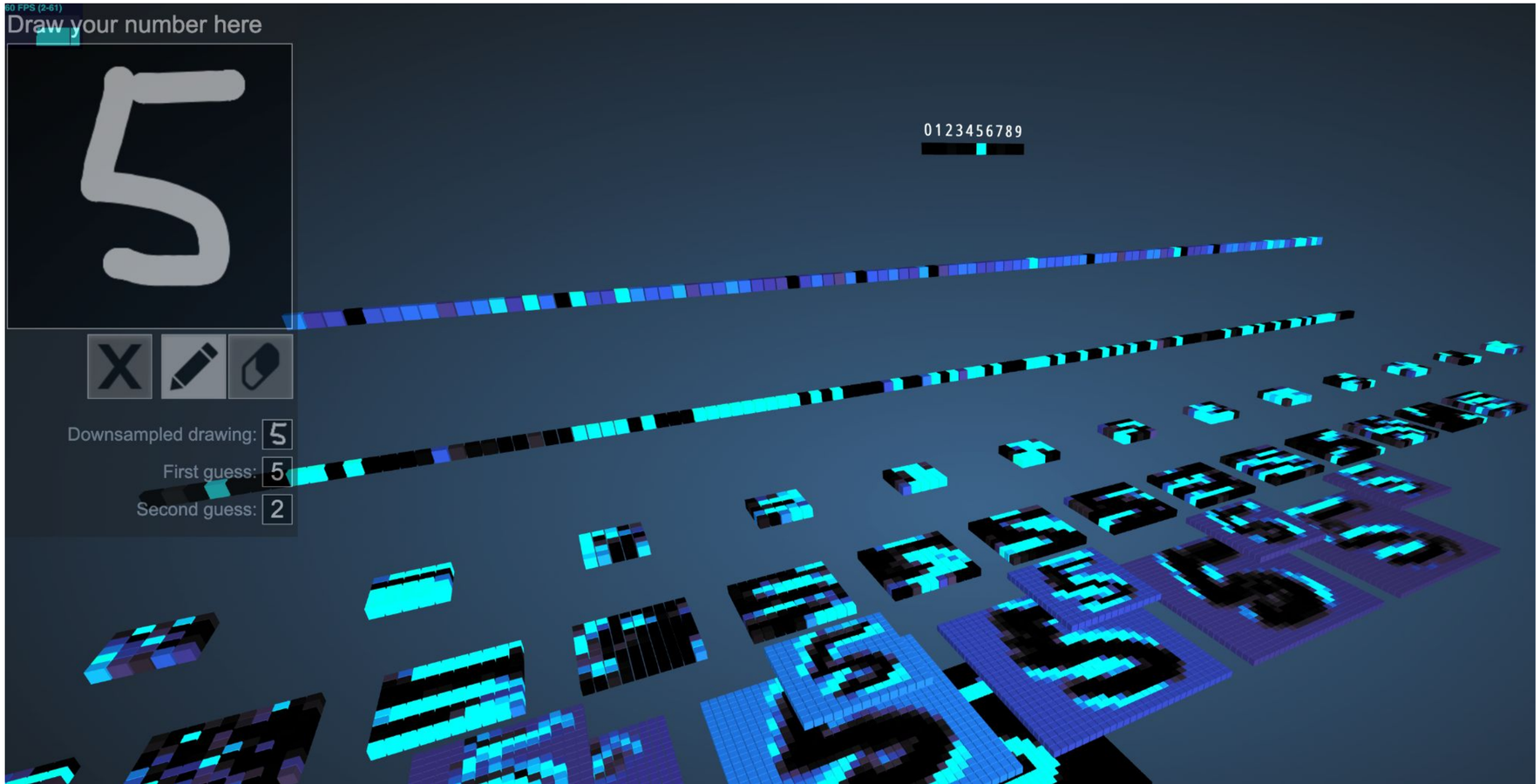




MNIST demo

3D demo: https://adamharley.com/nn_vis/cnn/3d.html

2D demo (lightweight): https://adamharley.com/nn_vis/cnn/2d.html

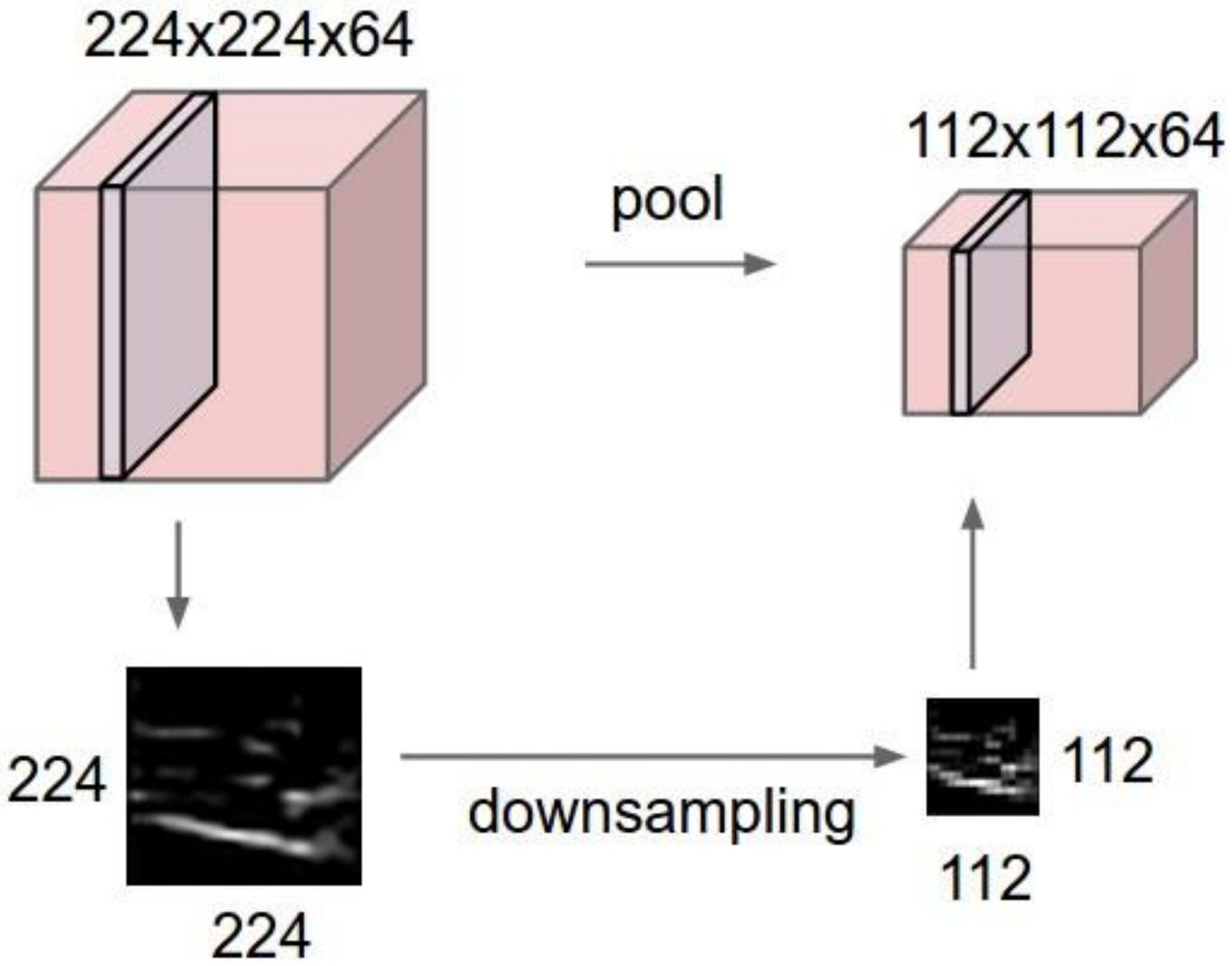


Pooling layer

girafe ai

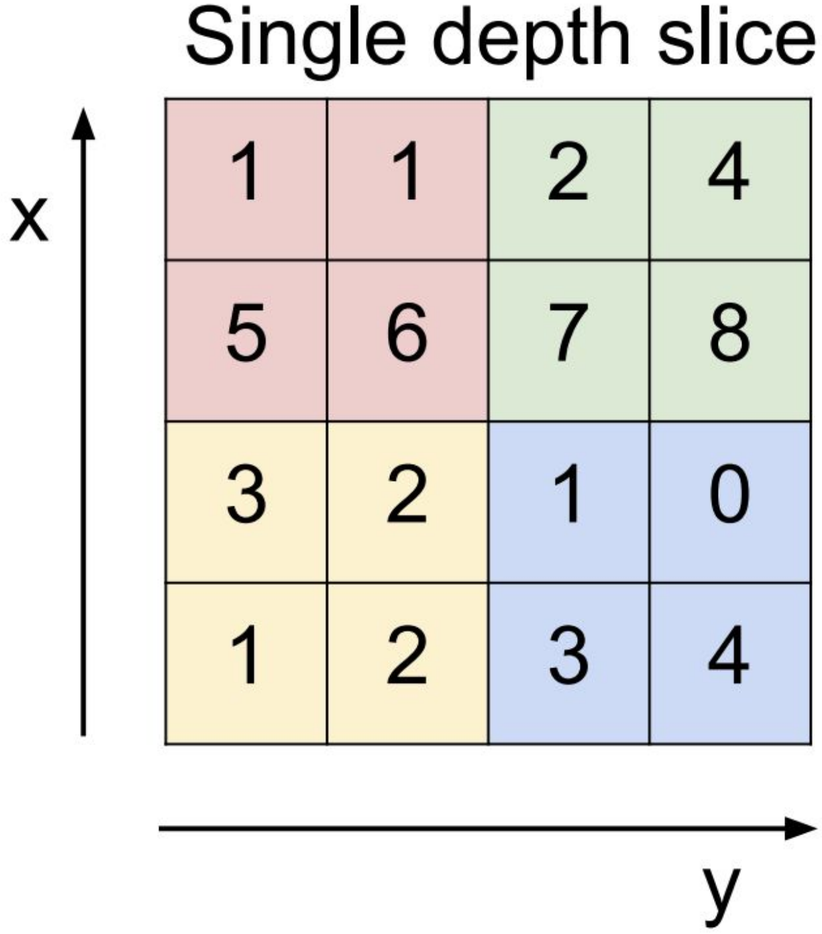
03

Pooling layer



- Makes the representations smaller and more manageable
- Operates over each channel independently
- No trainable params, so doesn’t overfit

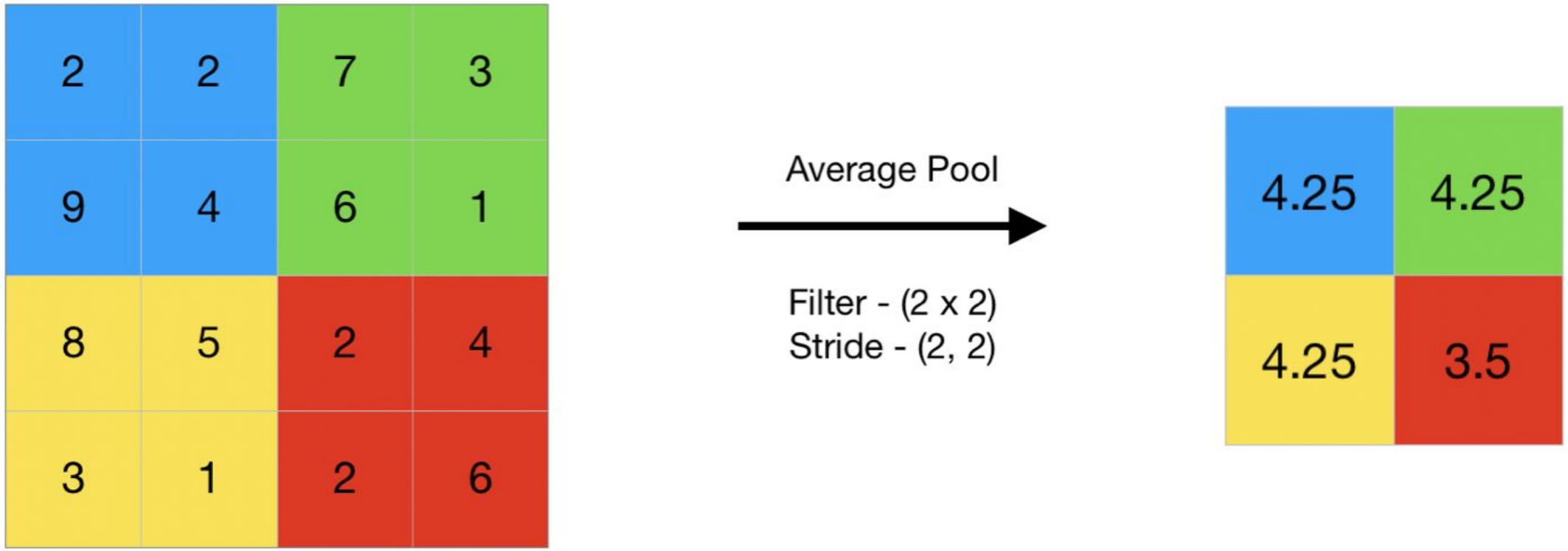
Max pooling



max pool with 2x2 filters
and stride 2

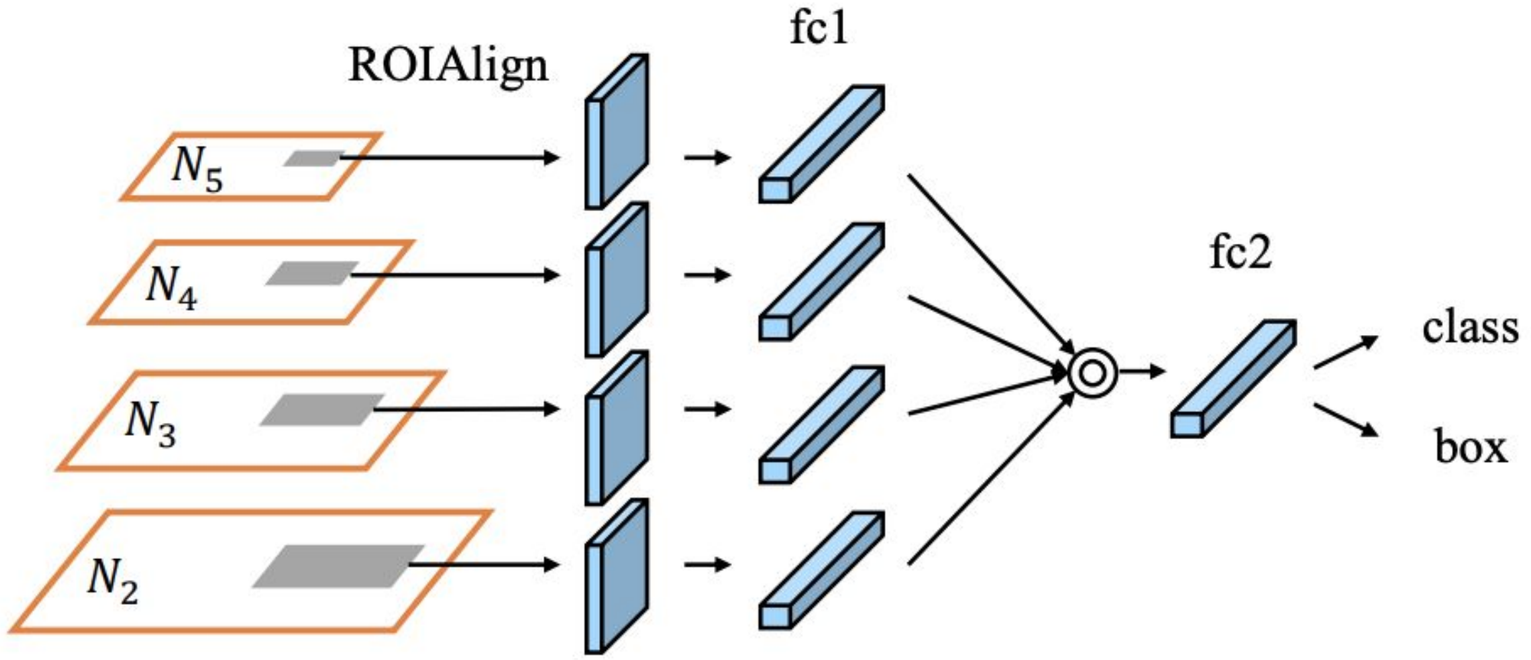
6	8
3	4

Average pooling



Adaptive pooling

- Pools adaptive region size into fixed output size
- Fixed size is required for following linear layers
- Allows to put arbitrary image size into convolution network
- Both Max- and Average-



[torch.nn.AdaptiveAvgPool2d]

Imagenet dataset

girafe ai

04

Image classification then

2007: Asirra

CAPTCHA that asks users to identify cats out of a set of 12 photographs of both cats and dogs.
Solved by **humans in 99.6%** of cases.
“Barring a major advance in machine vision, we expect **computers** will have no better than a **1/54,000 chance** of solving it”

Elson et al



Image classification now

2014: Asirra on kaggle

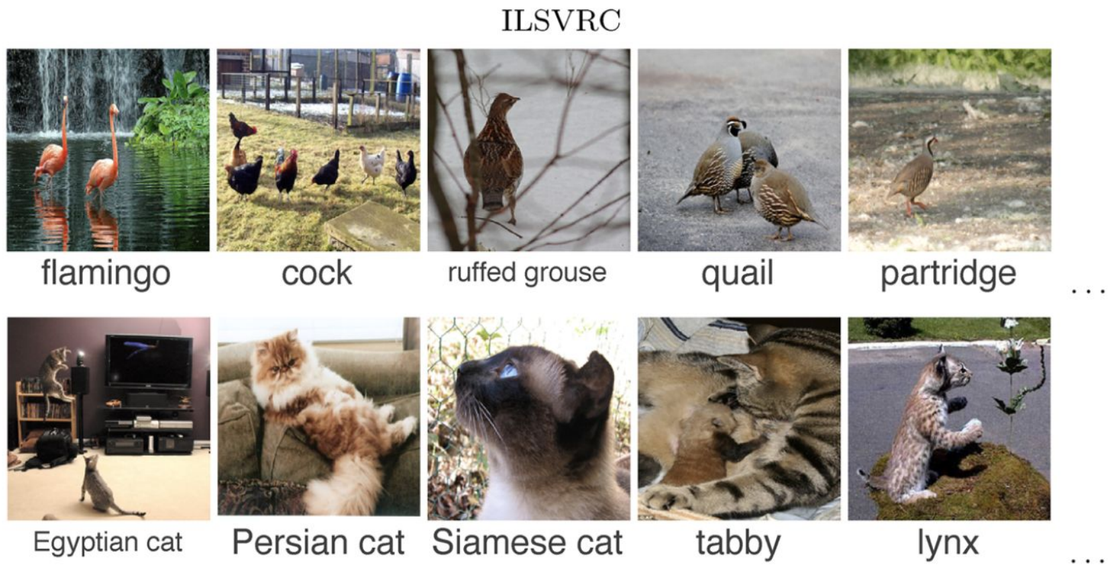
Competition page

#	△	Team	Members		Score	Entries	Last	Solution
1	—	Pierre Sermanet			0.98914	5	10y	
2	▲ 4	orchid			0.98308	17	10y	
3	—	Owen			0.98171	15	10y	
4	—	Paul Covington			0.98171	3	10y	
5	▼ 3	Maxim Milakov			0.98137	24	10y	
6	▼ 1	we've been in KAIST			0.98102	8	10y	
7	▲ 1	Doug Koch			0.98057	6	10y	
8	▲ 2	fastml.com/cats-and-dogs			0.98000	6	10y	

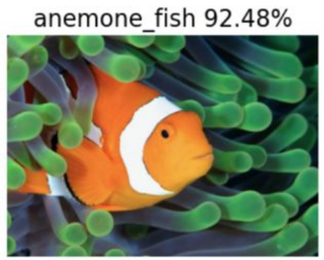
Last Submission: 02/02/2014, 1:43:19 GMT+4

IMAGENET competition

- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) since 2010
- 14.2m images of 244*244 pixels
- 1000 classes
- http://image-net.org/explore

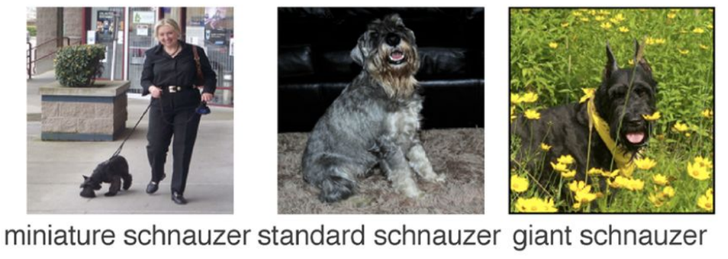














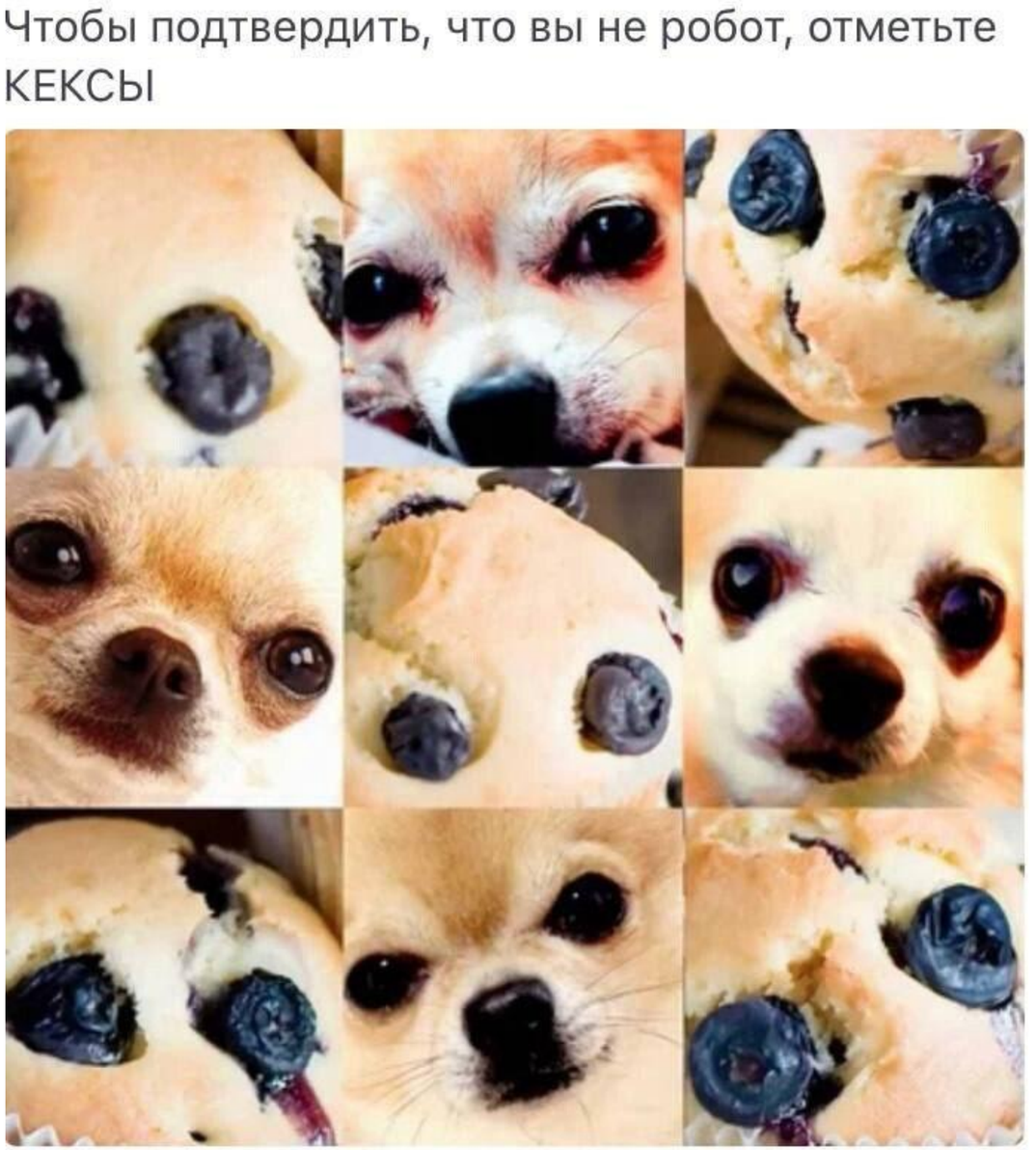






IMAGENET dataset

- Most of classes are dogs
- Most misclassified classes are chihuahua and cupcake

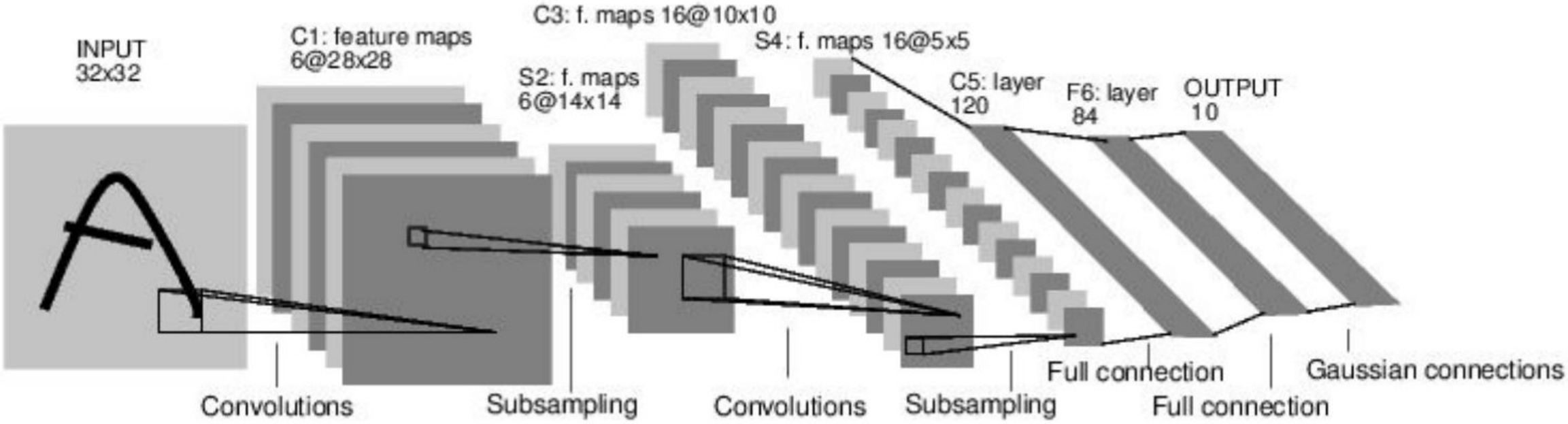


Classification architectures overview

girafe ai

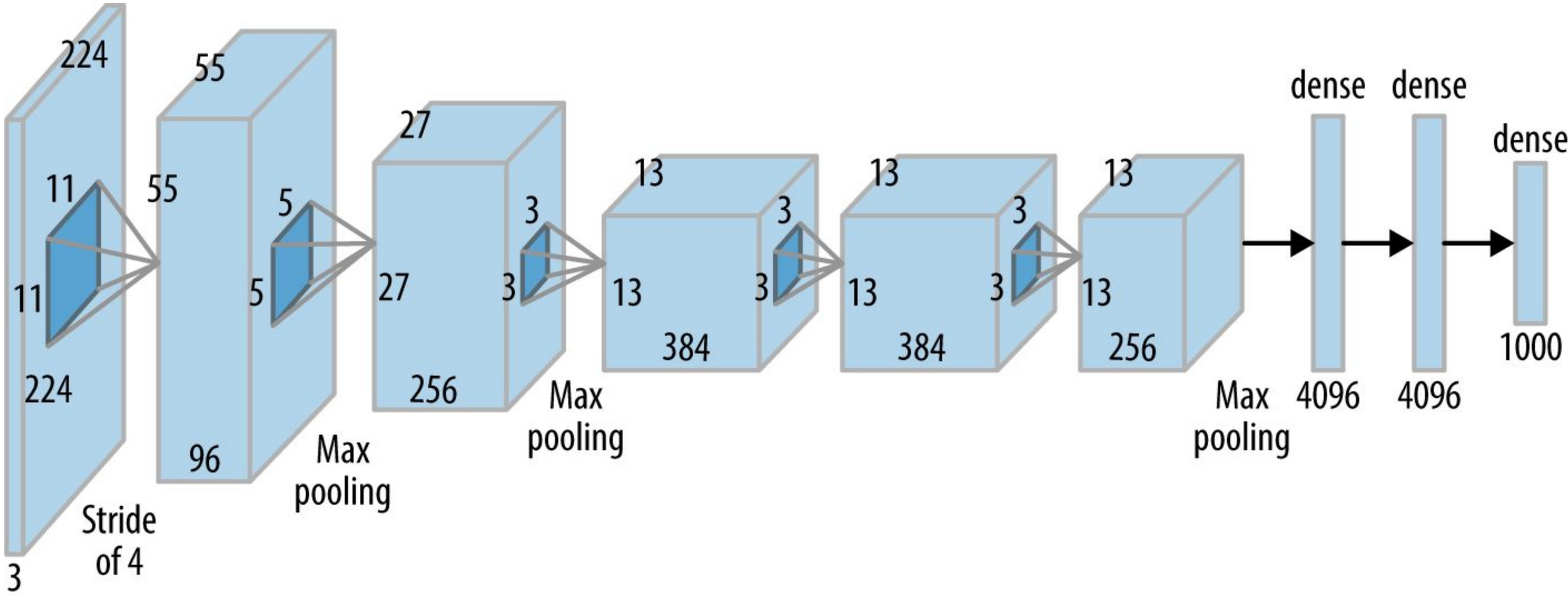
05

LeNet-5



[LeNet-5, LeCun 1998]

AlexNet

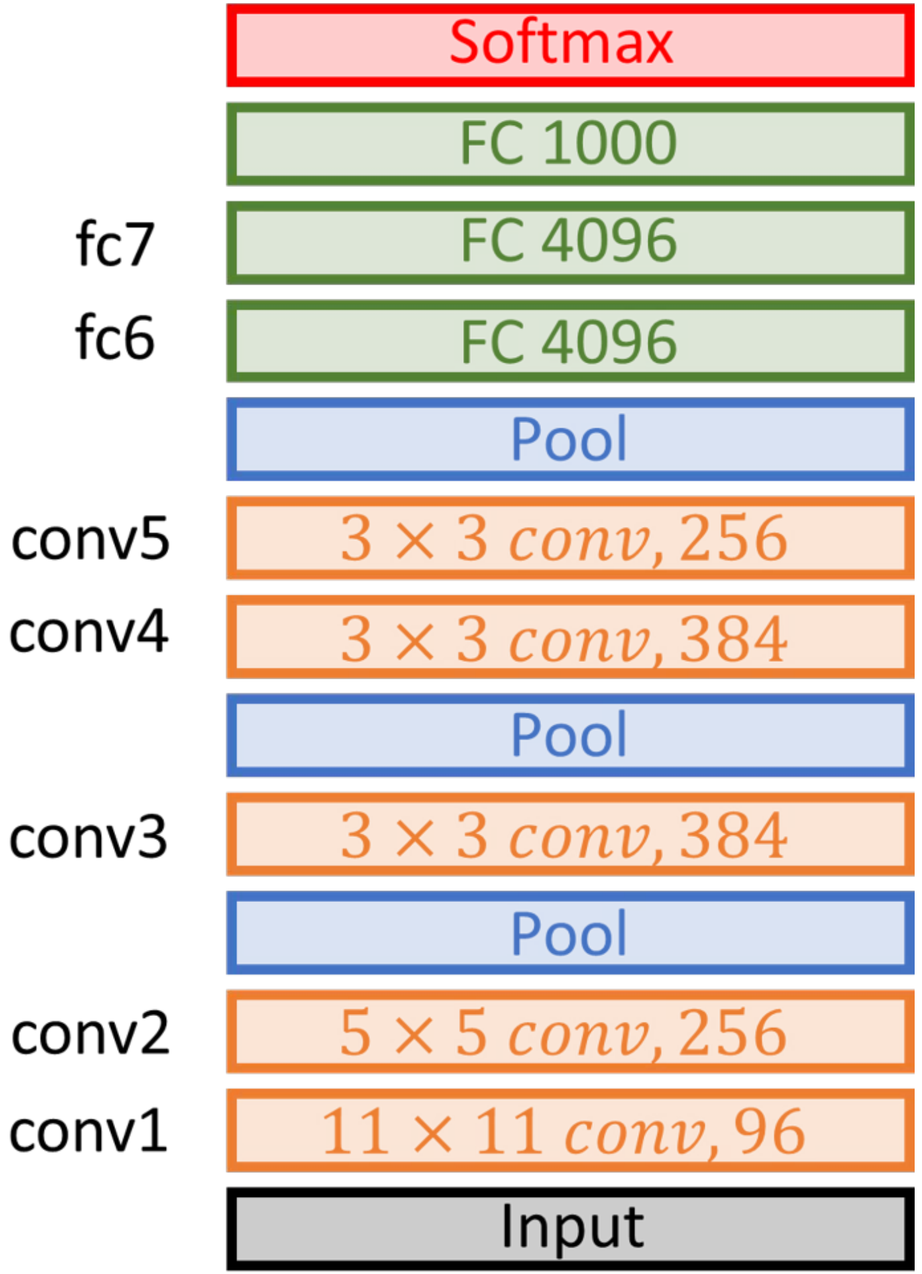

224
224
11
11
3
Stride
of 4
55
55
96
5
5
Max
pooling
27
27
256
3
3
Max
pooling
13
13
384
3
3
13
13
384
3
3
13
13
256
Max
pooling
dense
4096
dense
4096
dense
1000

AlexNet

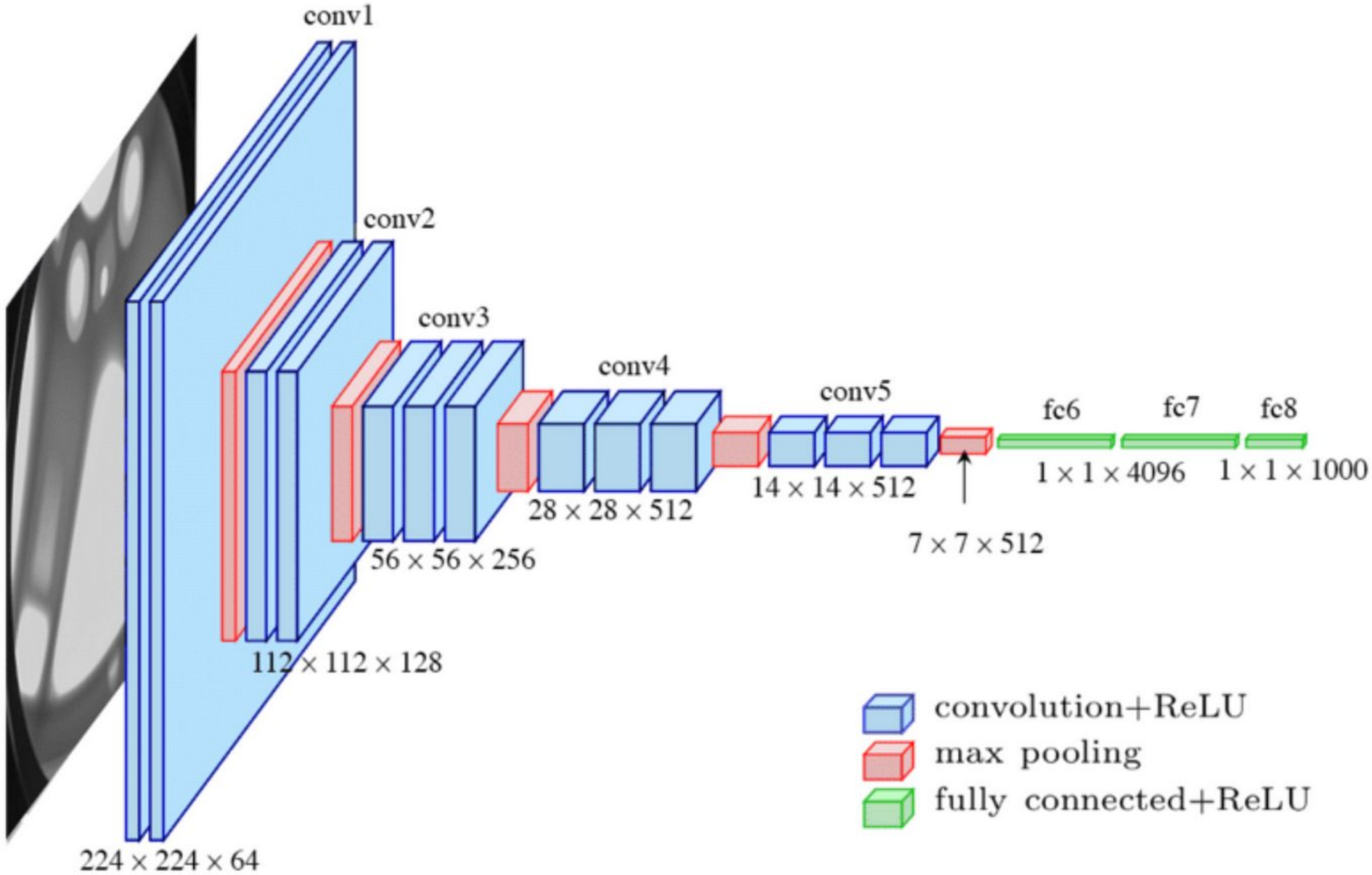
- Popularized use of ReLU
- Popularized use of GPU
- 15.3% top-5 error on LSVRC-2012

- Krizhevsky et al. 2012
- First win of ImageNet competition with Deep Neural Networks
- 7 layers, 60M parameters

paper, wikipedia



VGGNet


conv1
conv2
conv3
conv4
conv5
fc6
fc7
fc8
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU

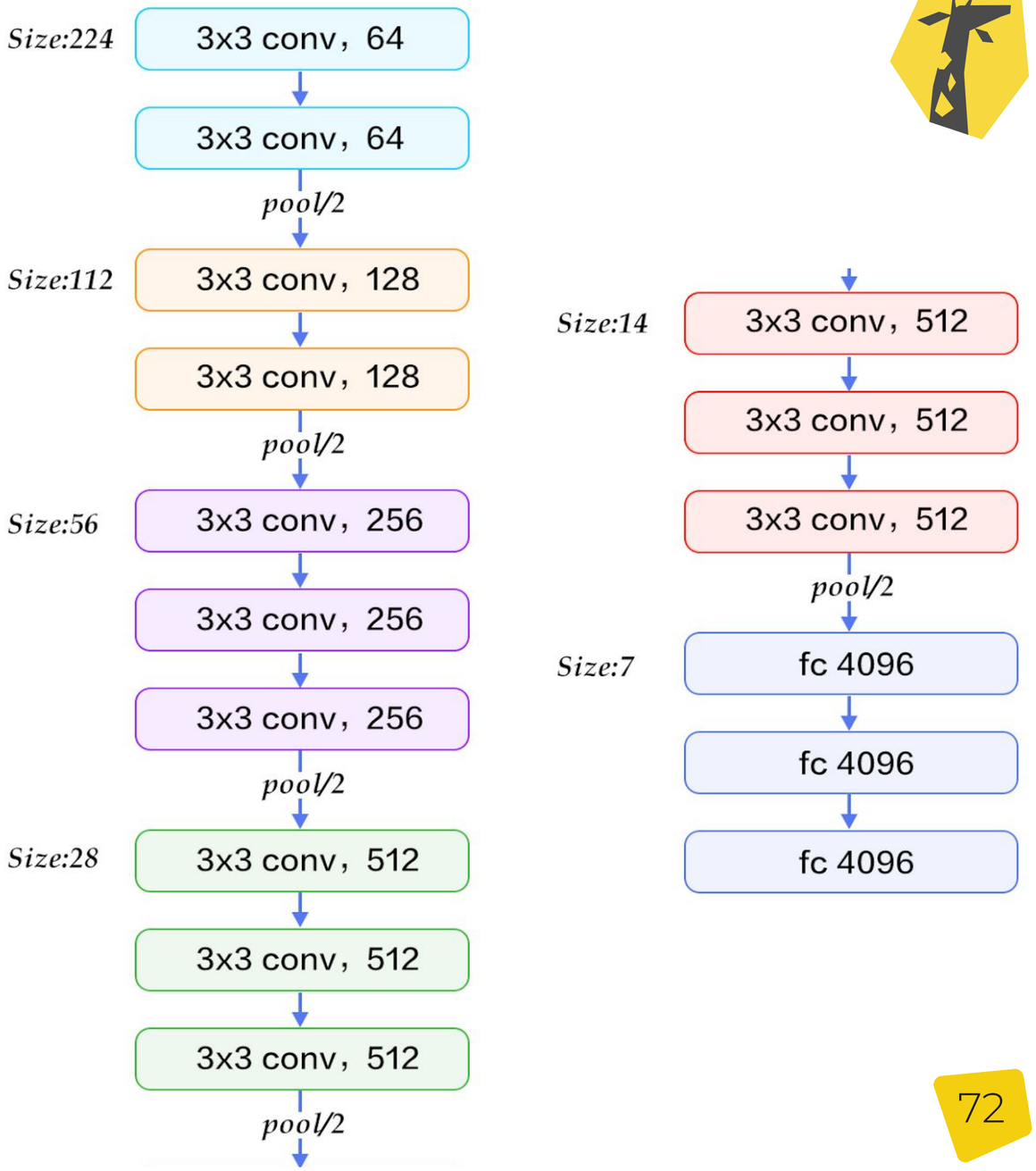
VGGNet

- All convolutions are 3x3
- Exponentially increasing channels
- Gradient vanishes =(

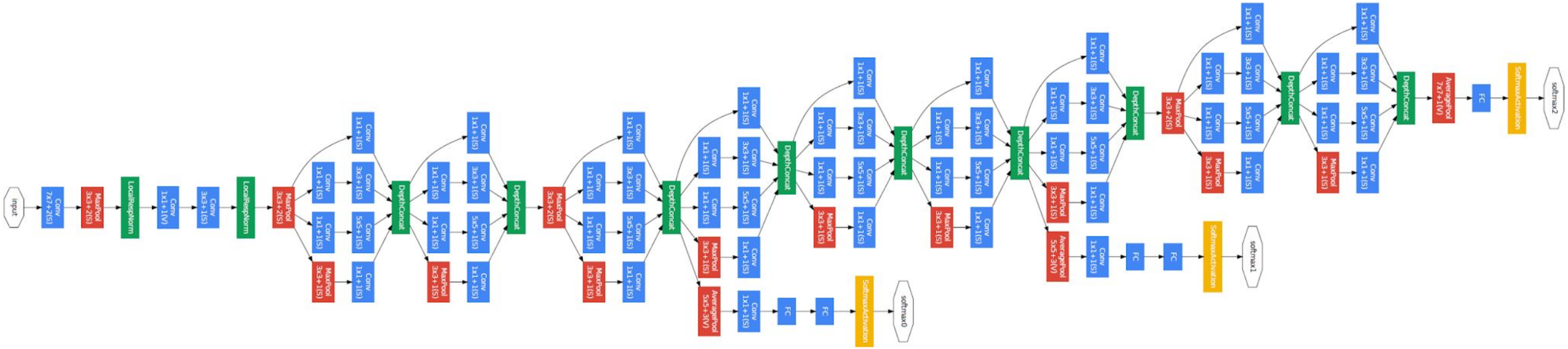
- 19 layers, 144M parameters
- Second place in 2014 Imagenet

https://en.wikipedia.org/wiki/VGGNet

https://arxiv.org/abs/1409.1556



GoogLeNet (aka Inception)





Inception module

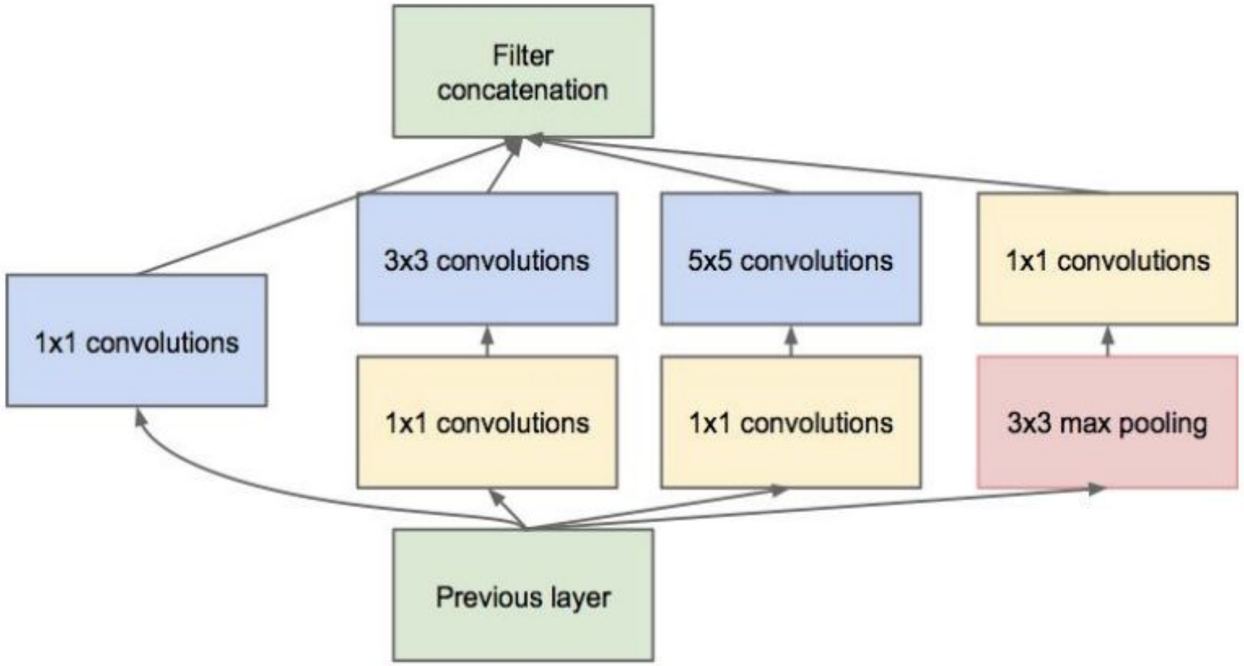
[Szegedy et al., 2014]

Inception module

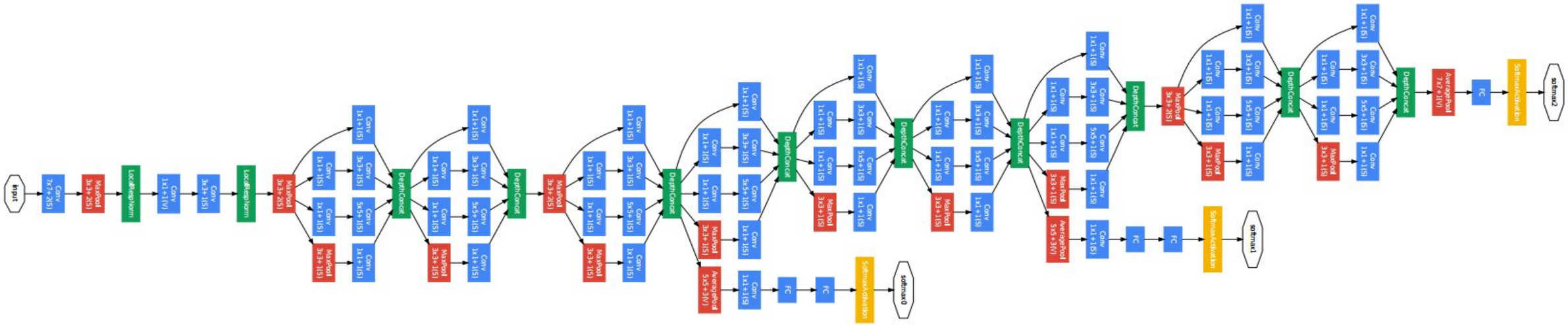
Multiscaling!

Each branch processes featuremap with different receptive field.

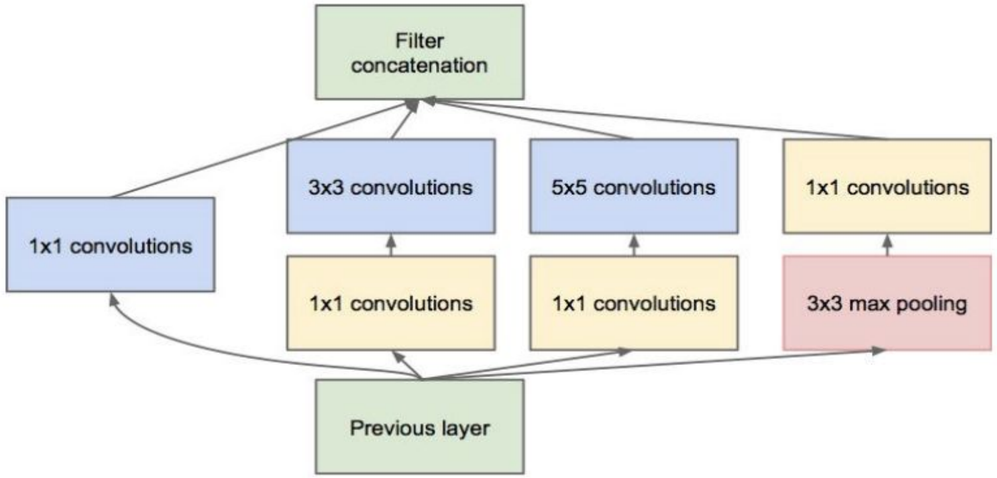
Concatenation of branch outputs allows to make scales even more variable on the next stage.



GoogLeNet



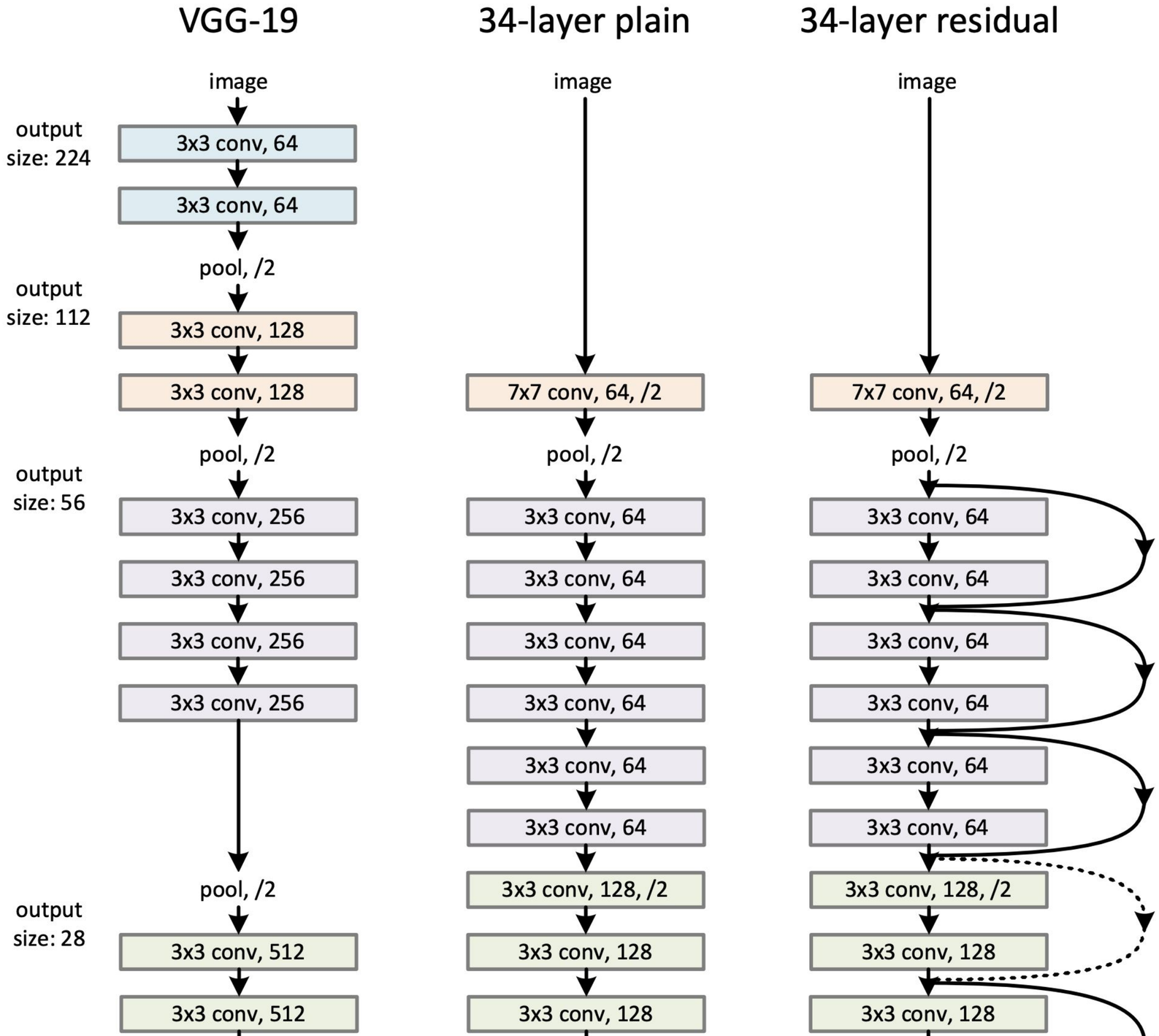
[Szegedy et al., 2014]



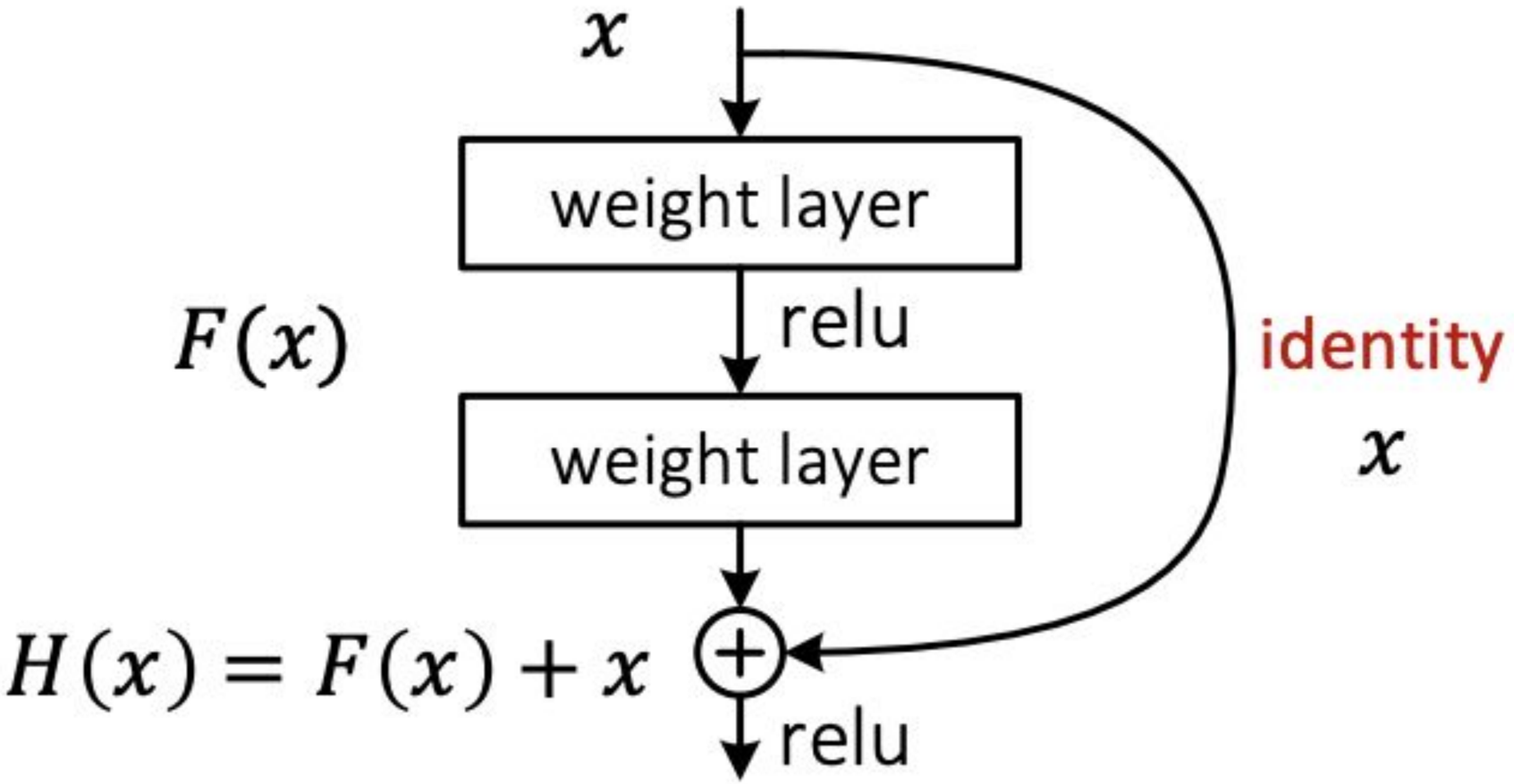
Inception module

ILSVRC 2014 winner (6.7% top 5 error)

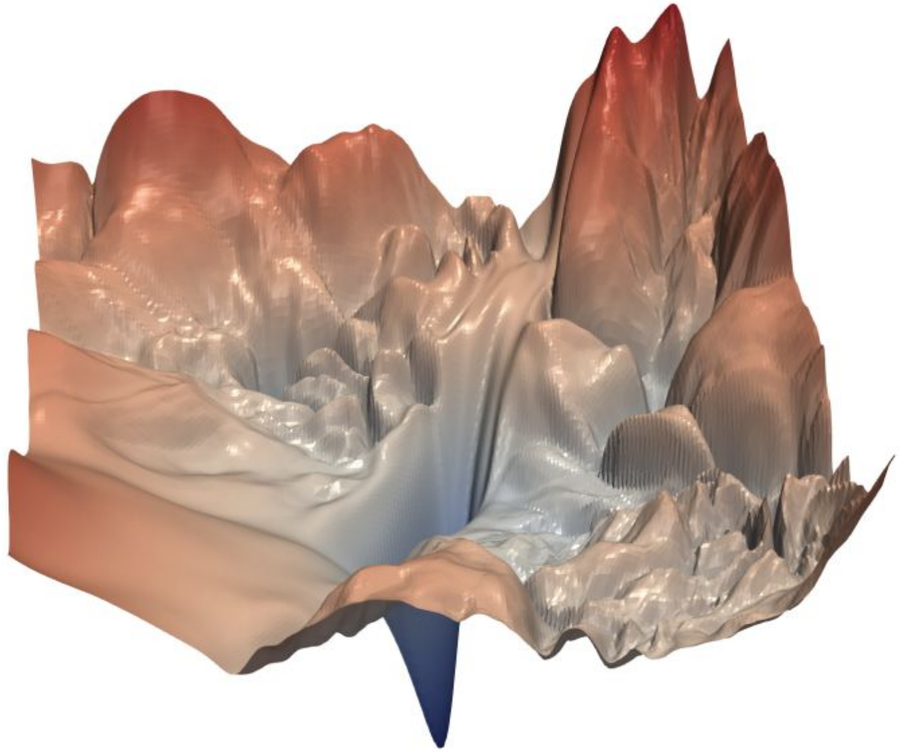
ResNet


VGG-19
34-layer plain
34-layer residual
image
output size: 224
3x3 conv, 64
3x3 conv, 64
pool, /2
output size: 112
3x3 conv, 128
3x3 conv, 128
pool, /2
output size: 56
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
pool, /2
output size: 28
3x3 conv, 512
3x3 conv, 512
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 128
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 128

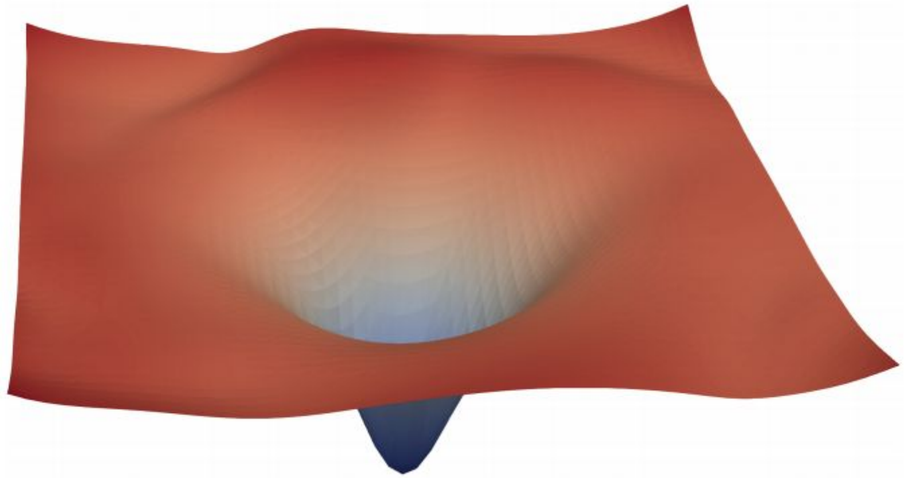
Residual block



Loss influence of residuals



(a) without skip connections



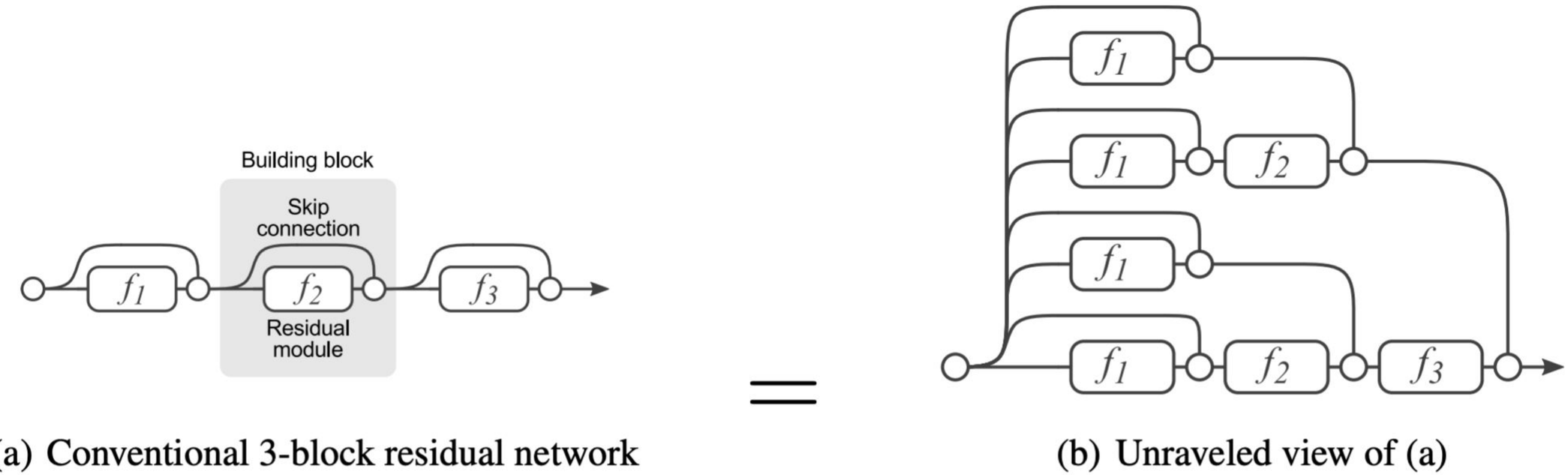
(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

NIPS 2018

ResNet is implicit ensemble

Residual Networks Behave Like Ensembles of Relatively Shallow Networks



(a) Conventional 3-block residual network

(b) Unraveled view of (a)

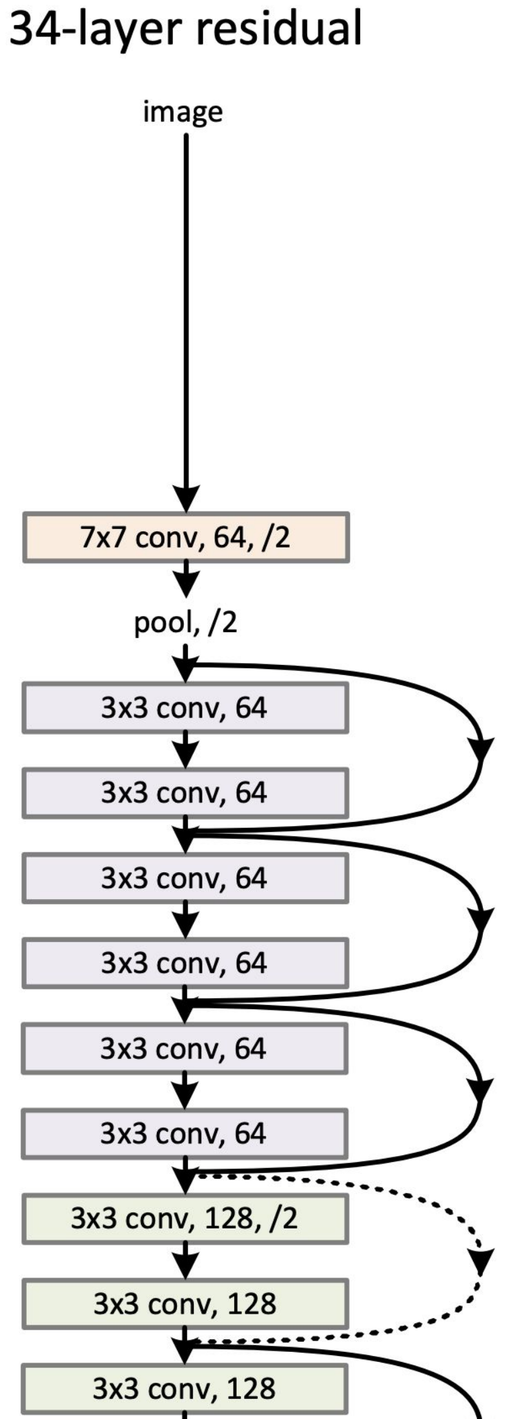
Paper: https://arxiv.org/pdf/1605.06431.pdf

ResNet summary

- Invented Residual connection
  - preserves gradient
- Batch Normalization after every conv layer
- No dropout used
- 151 layers
- ImageNet 2015 winner
  - 3.6% top 5 error

https://doi.org/10.1109/CVPR.2016.90

https://arxiv.org/pdf/1512.03385.pdf



ResNeXt and co

Same as ResNet, but scalable

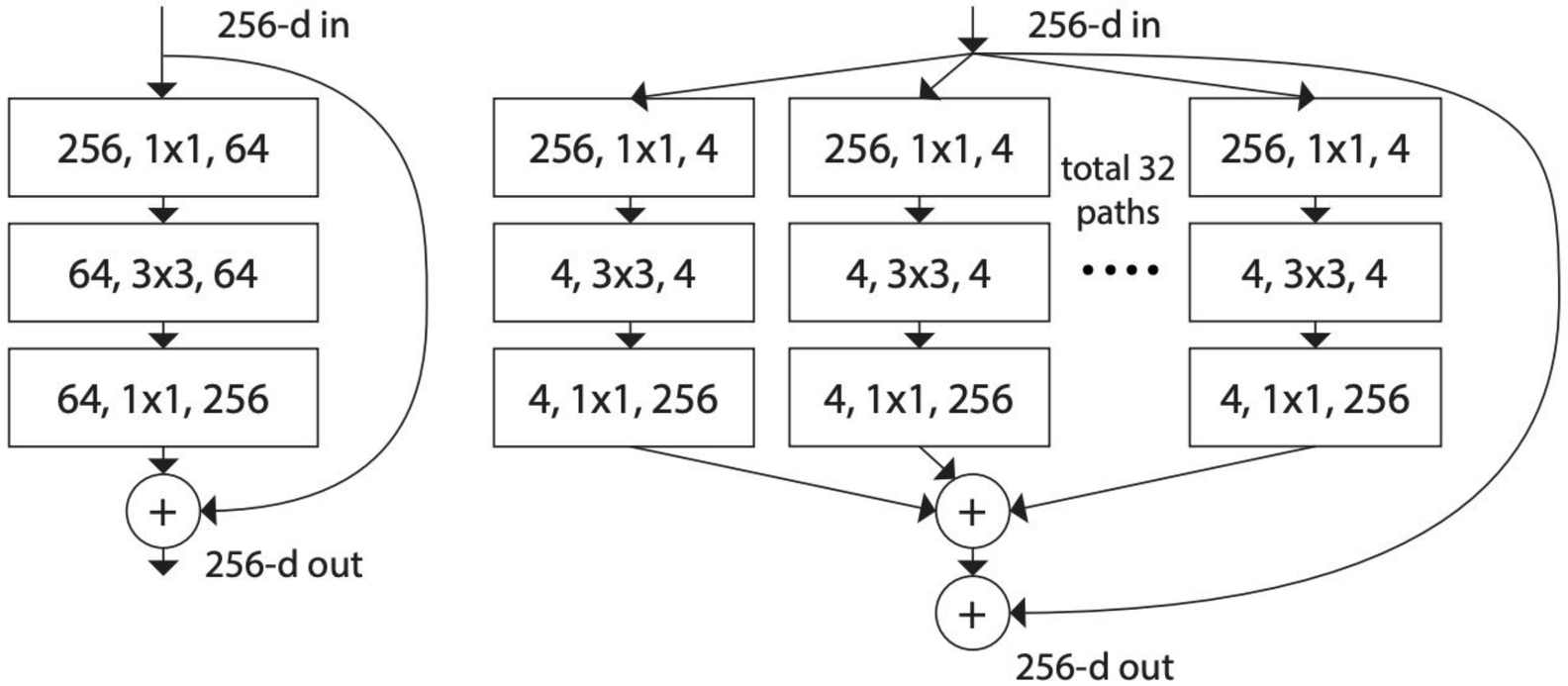
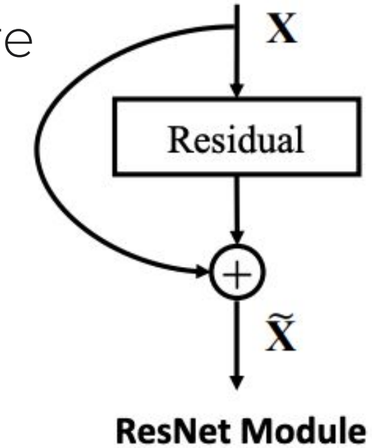


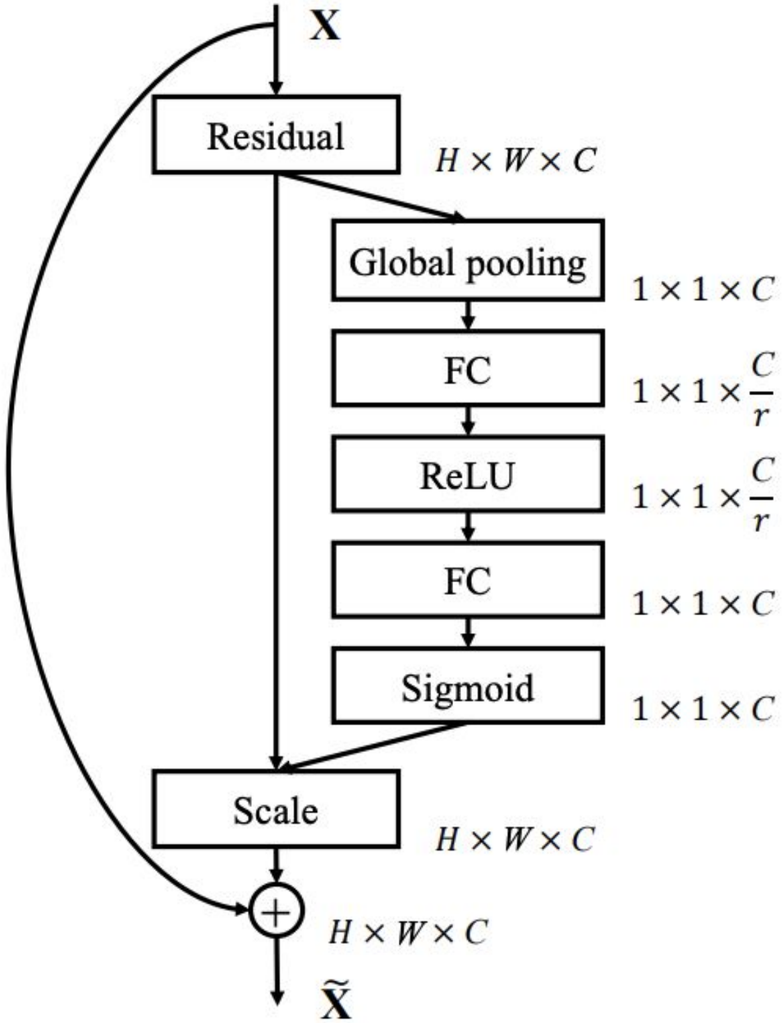
Figure 1. **Left**: A block of ResNet [14]. **Right**: A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

Squeeze-and-Excitation block

1. Let's improve ResNet architecture with 1x1 conv after main transformations
2. SE block behaves as 1x1 conv (rescaling channels), but with a twist
   a. It implements autoencoder concept: first compresses information and then return to original dimensionality
   b. behaves like **"non-linear PCA"**



ResNet Module



SE-ResNet Module

https://arxiv.org/pdf/1709.01507

Segmenation:
https://arxiv.org/pdf/1804.03999

SE block is attention mechanism

Linear layer (no attention):

$\mathbf{Z} = \mathbf{X} * W + b$

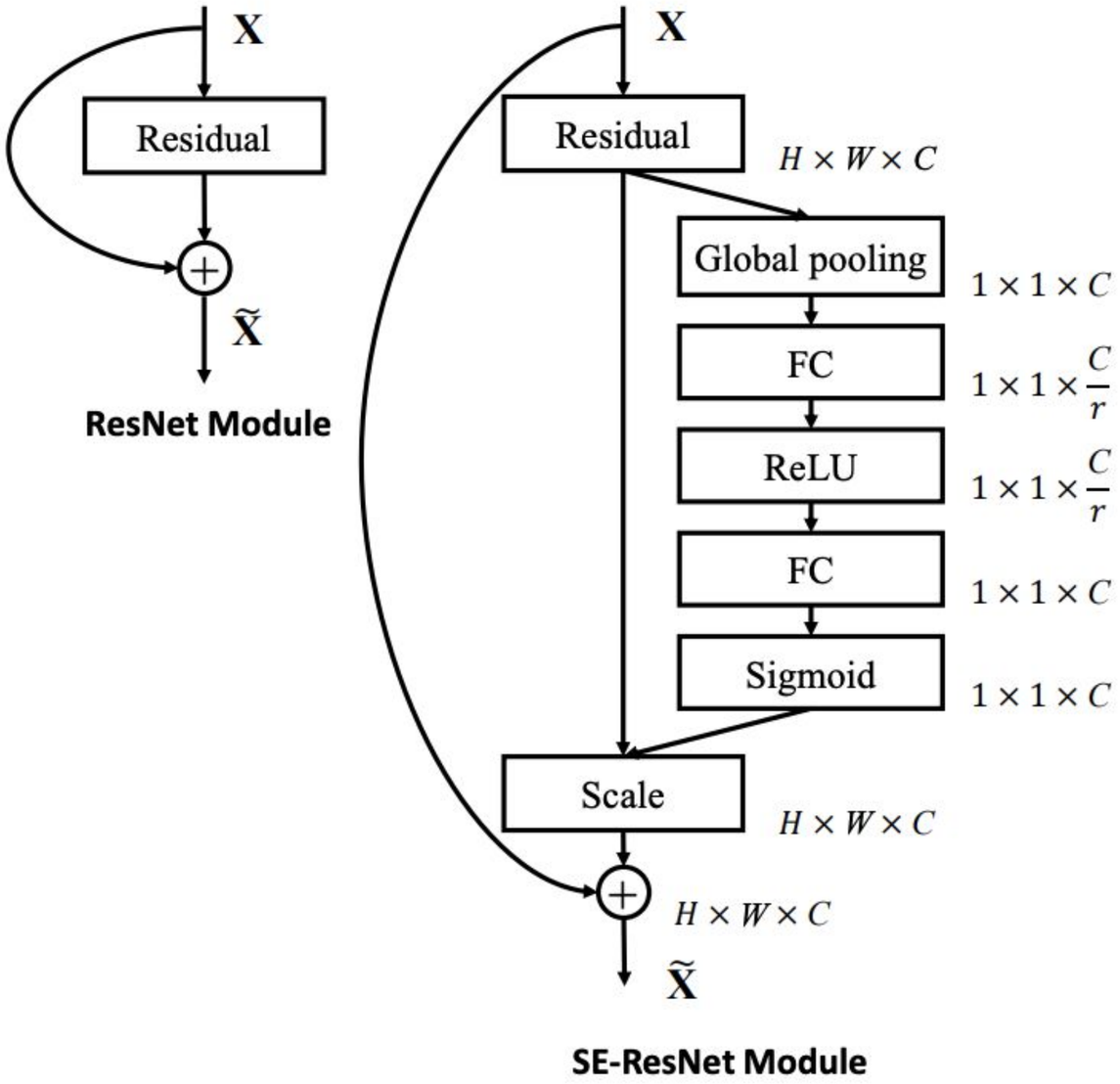
W, b - parameters

Linear layer with attention:

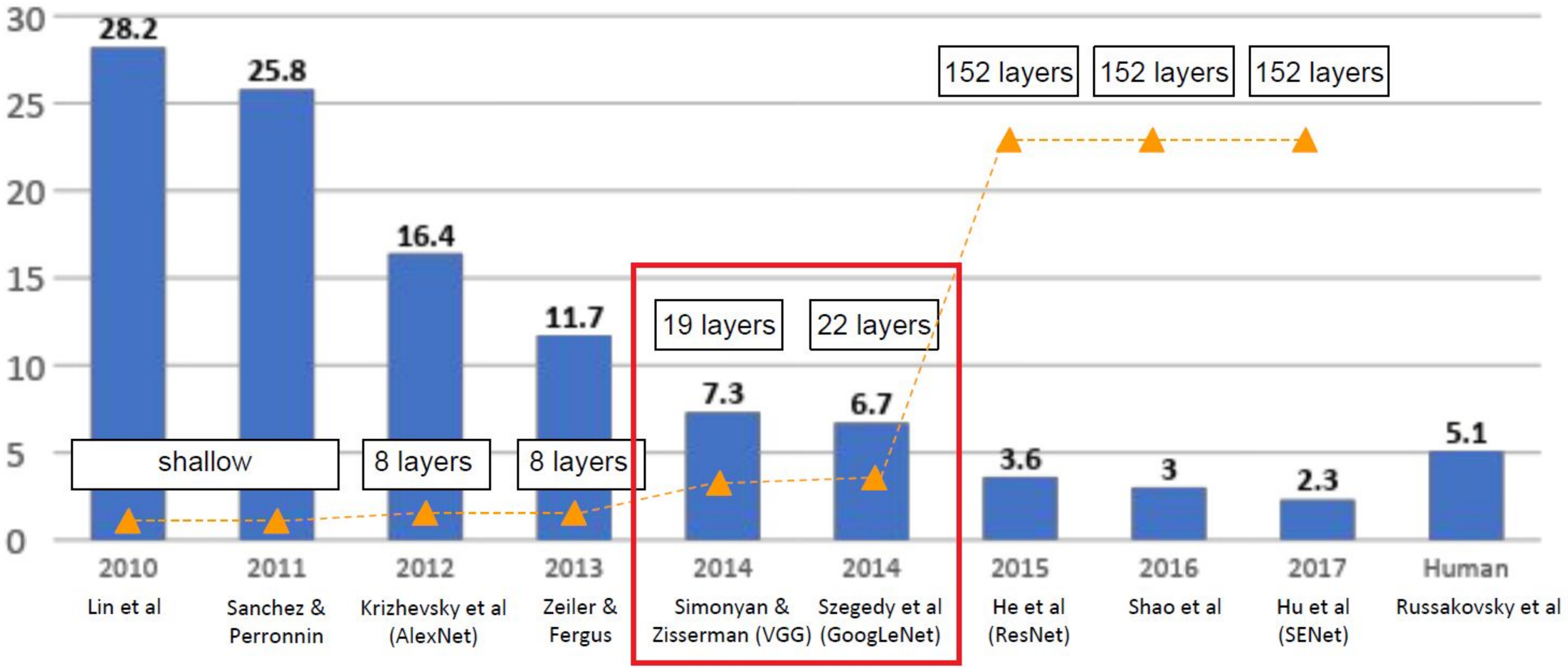
$\mathbf{Z} = \mathbf{X} * W_h(\mathbf{X}) + b_h(\mathbf{X})$

W, b - functions of **X**,
parametrized by h

h - parameters



ImageNet summary



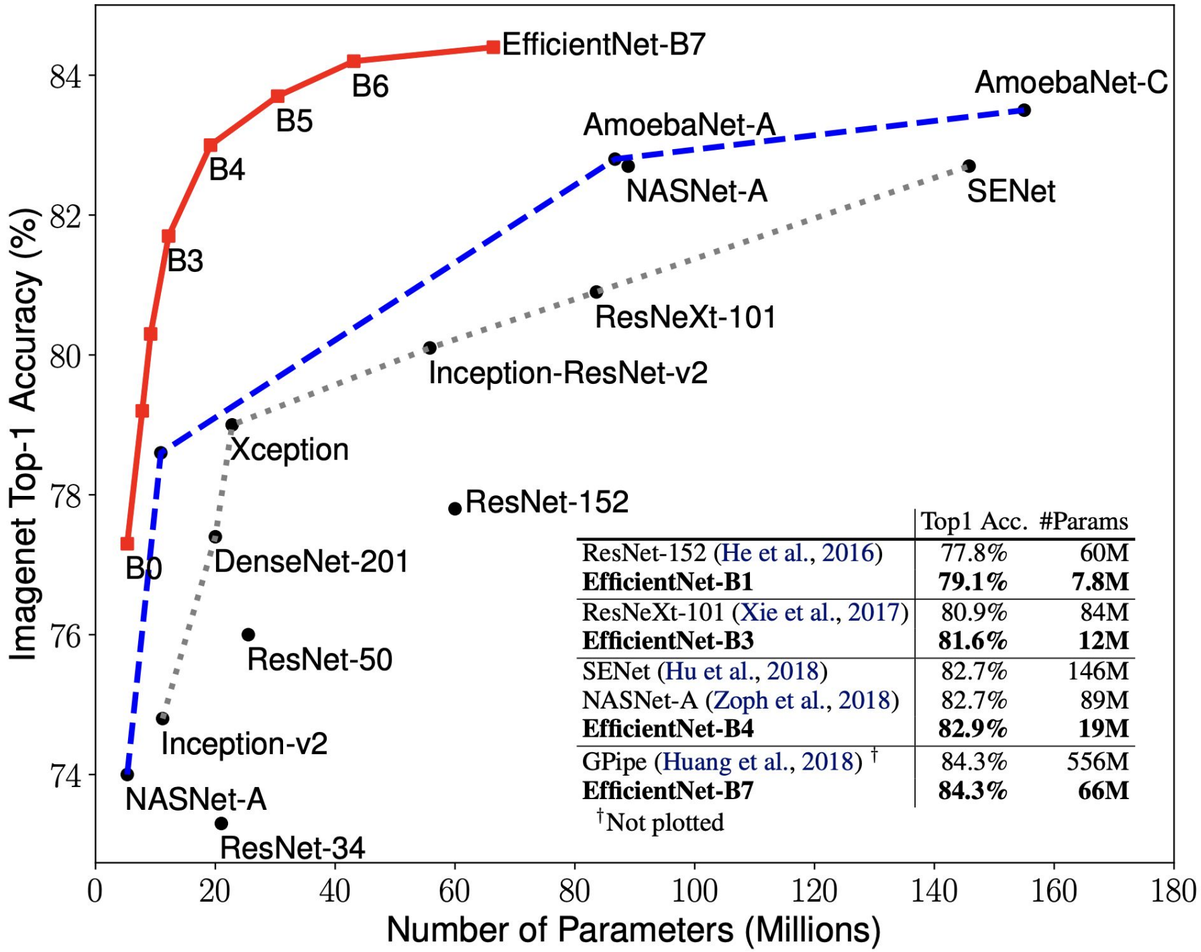
Contemporary CNN architecture

can be represented by **EfficientNet**.

It has beats SENet and is smaller and more *efficient*.

Don’t use ResNet as basic architecture today!!!
It is hopelessly outdated…

https://arxiv.org/pdf/1905.11946



	Top1 Acc.	#Params
ResNet-152 (He et al., 2016)	77.8%	60M
EfficientNet-B1	**79.1%**	**7.8M**
ResNeXt-101 (Xie et al., 2017)	80.9%	84M
EfficientNet-B3	**81.6%**	**12M**
SENet (Hu et al., 2018)	82.7%	146M
NASNet-A (Zoph et al., 2018)	82.7%	89M
EfficientNet-B4	**82.9%**	**19M**
GPipe (Huang et al., 2018) †	84.3%	556M
EfficientNet-B7	**84.3%**	**66M**

†Not plotted

EfficientNet scaling

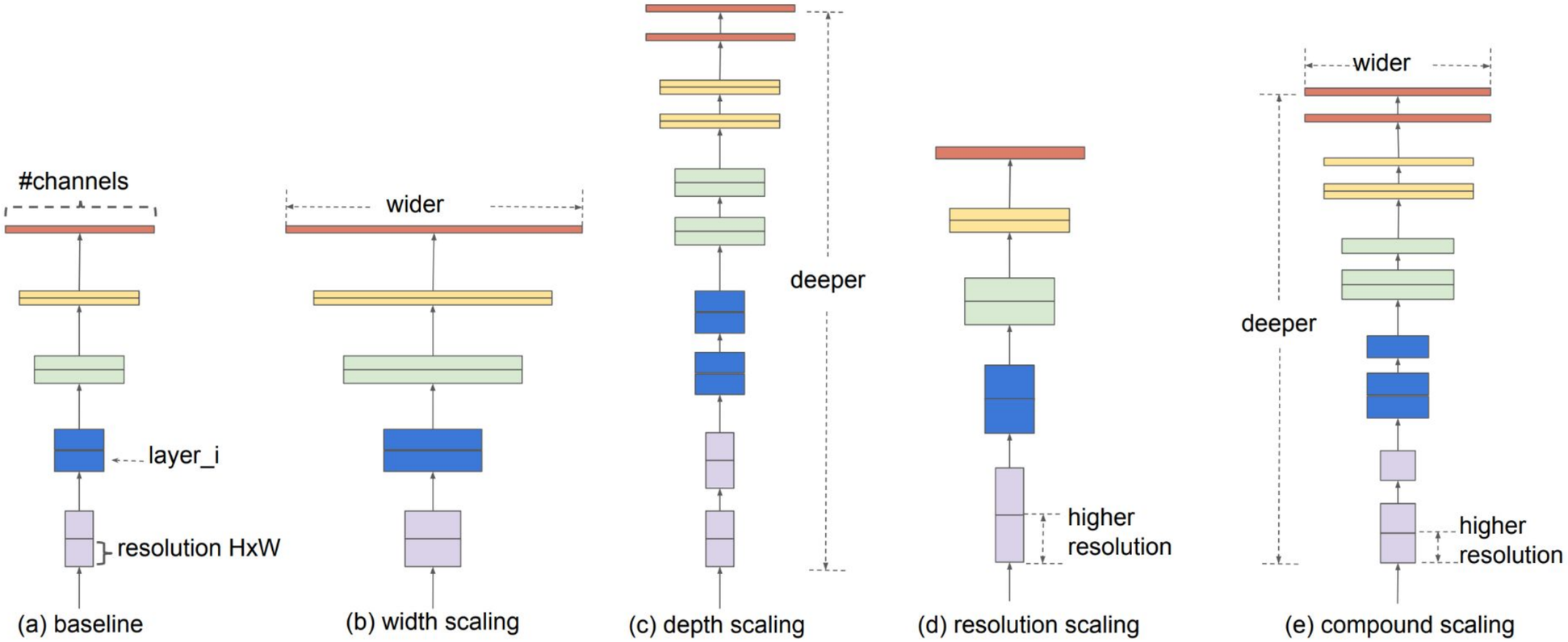
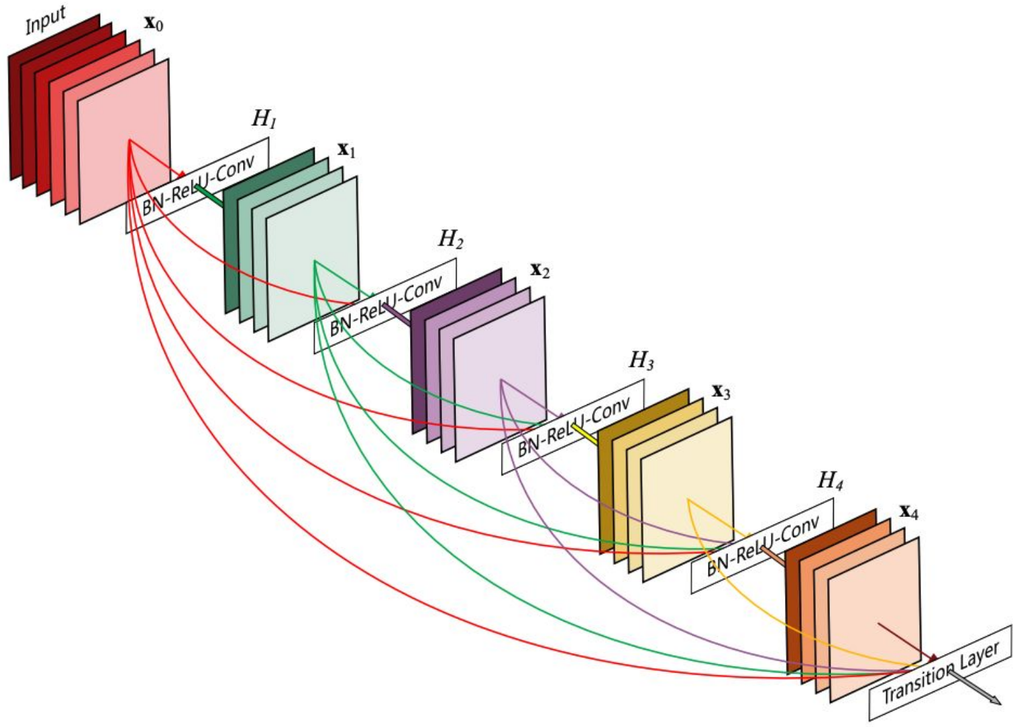


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Other approaches to save gradient

Vanishing gradient is present in **all** deep neural network architectures.

- Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small during backpropagation
- Lower levels are hard to train and are trained slower
- Solutions
  - skip-connections (ResNet)
  - dense connections (DenseNet)



Summary



- ConvNets stack convolutional, pooling and dense layers
- Convs trend towards smaller filters (3x3) and deeper architectures
- 1x1 convolutions are useful
- Humanity is already beaten on ImageNet
- Use contemporary architectures!

Revise



1. Convolutional layer
    a. hyperparameters
    b. receptive field
    c. 1x1 conv
    d. memory consumption
2. CNN filter analysis
3. Pooling layers
    a. Adaptive poolings
4. Classification architectures overview
    a. LeNet
    b. AlexNet
    c. VGG
    d. Inception
    e. ResNet
    f. SENet
    g. EfficientNet

Thanks for attention!

Questions?



girafe
ai

